Kirinyaga University

Mobile: +254 0709 742000, P.O.Box: 143-10300 Kerugoya.
Email: info@kyu.ac.ke Web: www.kyu.ac.ke

SCHOOL: [Insert School]
DEPARTMENT: [Insert Department]
PROGRAMME: [Insert Programme]
YEAR: [Insert Year] **SEMESTER:** [Insert Semester]
UNIT CODE: COM 312 **UNIT TITLE:** System Programming
LECTURE HOURS: 42 (3 hrs/week) **PRE-REQUISITES:** [Insert Pre-requisites]
LECTURER'S NAME: Mr. Samuel Maina
LECTURER'S CONTACTS: TEL: [Insert Tel]
EMAIL: samicho4@gmail.com

Welcome Message

Welcome to COM312: System Programming. This course introduces you to programming operating-system modules, giving you a working command of process control, device and signal handling, memory and file management, and inter-process/inter-machine communication using C/C++.

Learning Outcomes

At the end of the course the learner will be able to:

- Explain the scope and tasks of systems programming
- Describe processes as fundamental objects in the OS
- Describe the basic concepts of threads and multithreaded programming
- Describe system calls for files and use programming operations for files and directories

Teaching Methodology

- Stand-up instruction (lectures)
- Presentations and guided discussions
- Tutorials
- Individual and group exercises
- Practical-oriented exercises

Instructional Materials

- Whiteboards
- LCD projectors
- Computers
- Resource persons

Course Evaluation

- CAT 1 = 10% | CAT 2 = 10% | Assignments = 10%
- End of Semester Examination = 70% (Total = 100%)
- Pass mark: 40%

COURSE OUTLINE

S/N.	Week	Sub-Topic	Remarks
1.	**An Introduction to Systems Programming and Essential Concepts**	• Design principles of Linux • Overview of systems programming and objectives • Components of systems programming • Theory and practical of system calls using C	3 hrs
2.	**Preparing Your Tool Box**	• GNU gcc and gdb • NASM (Netwide Assembler) • UNIX readelf, objdump, od, nm, ldd and ld config • Creating your own static and dynamic libraries	3 hrs
3.	**UNIX File System**	• Linux directory structures • Basic shell scripting • UNIX and POSIX file systems • Inodes, APIs, and UNIX kernel support for files	3 hrs
4.	**CAT I**	• Contents covered	3 hrs
5.	**Process Management and Scheduling**	• Process identification • Process trees, chains and fans • Zombie and orphan processes • Process groups, sessions and controlling terminals • UNIX daemons • System V R3 and System V R4 schedulers	3 hrs
6.	**UNIX Inter-Process Communication Tools**	• Signals • Pipes / FIFOs • Message queues • Shared memory • Memory mapping	3 hrs
7.	**Thread Management and Synchronization**	• POSIX thread API • Introduction to threads • Synchronization among threads using mutexes and condition variables • Synchronization among related and unrelated processes using semaphores	3 hrs

S/N.	Week	Sub-Topic	Remarks
8.	**Network Programming**	• UNIX socket API • Writing concurrent servers – multi-process model • Writing concurrent servers – multi-threaded model • Non-blocking I/O using the select() system call	3 hrs
9.	**Network Security**	• Buffer overflow vulnerabilities • Exploiting vulnerabilities using Metasploit frameworks	3 hrs
10.	**Group Presentation Practicals (Assignment)**	• Identified area for group presentation: pipes practical	3 hrs
11.	**Group Presentation Practicals (Assignment)**	• Identified area for group presentation: network programming	
12.	**CAT 2**	• Contents covered	3 hrs
13.	**Linux User Management (Personal Reading Topic)**	• Linux users • Linux user management • User passwords • Local groups	3 hrs
14.	**Revision Week**	• Course revisions	3 hrs

COURSE TEXTBOOK

i) Stevens, W. R. Advanced Programming in the UNIX Environment.

REFERENCE TEXTBOOKS

i) Kerrisk, M. (2010). The Linux Programming Interface: A Linux and UNIX System Programming Handbook. No Starch Press.

ii) Raymond, E. S. The Art of UNIX Programming.

Systems programming lesson 1

COM 312: BSc Computer science/BSc IT

Topics area coverage.

- **Brief history of Linux**
- **Design principles of Linux**
- **Overview of what is systems programming**
- **Components of systems programming**
- **Theory and practical of systems call using C programing language.**

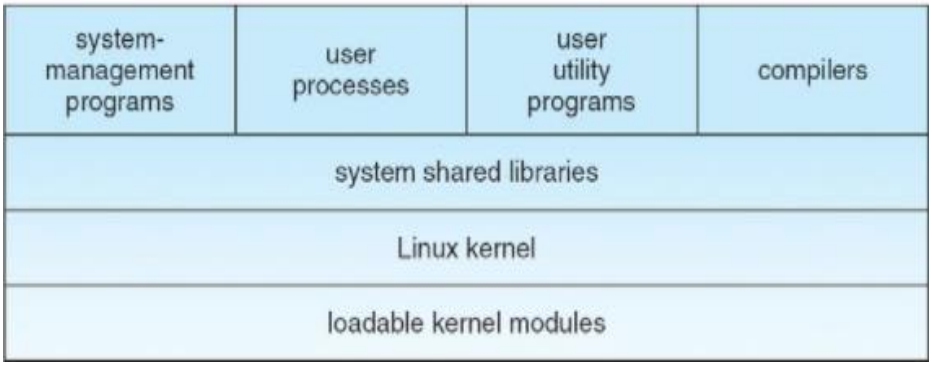
Brief history of Linux

- **Linux is a modern, free operating systems based on UNIX standards.**
- **First developed as a small but self -contained kernel in 1991 by Linus Torvalds with major design goal of UNIX compatibility.**

Design principles of Linux

- **Main design goals are speed, efficiency, and standardization**
- **Linux is a multi-user, multitasking systems with a full set of UNIX – compatible tools.**
- **Linux is designed to be compliant with the relevant POSIX documents (*portable operating system interface*) at least two Linux distributors have achieved official POSIX certification.**
- **The Linux programming interface adheres to the SVR4 UNIX semantics, rather than to BSD behavior**

COMPONENTS OF LINUX SYSTEMS



Linux system management - refers to the ability to manage Linux-based systems such as desktops, laptops, and servers

User processes- A Linux process is nothing but running instance of a program. For example, when you start Firefox to browse Internet, you can create a new process. In Linux, each process is **given a unique number** called as a process identification (PID)

User utility programs- Utility programs that **provide the user most of the functionalities of an operating systems**.

Compilers - computer software that translates (compiles) source code written in a high-level language (e.g., C++) into a set of machine-language instructions that can be understood by a digital computer's CPU.

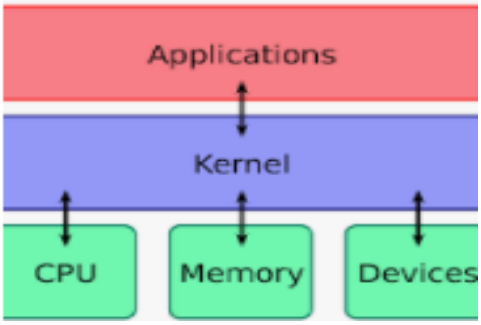
System shared libraries- Shared Libraries are **the libraries that can be linked to any program at run-time**. They provide a means to use code that can be loaded anywhere in the memory. Once loaded, the shared library code can be used by any number of programs

Components of A Linux System

- Linux system is composed of **three main bodies of code**, in the line with the most traditional UNIX implementations.
  i. Kernel
  ii. System libraries
  iii. System utilities

1. Kernel

The kernel is responsible for maintaining all the important **abstractions** of the operating systems, including such things as virtual memory and processes.



Kernel is an **interface** between application and actual data processing at the hardware level.

The kernel connects the system hardware to the application software.

- **The more internal processes are provided by the kernel**
- **Whenever a system starts, the kernel is the first program that is loaded after the bootloader because the kernel has to handle the rest of the thing of the system for the operating system.**

- **The kernel remains in the memory until the operating system is shut down.**

Typically, kernel is responsible for low –level task (or you can say the basic functions of kernel are):-

- **Access Computer resource:** A Kernel can access various computer resources like the CPU, I/O devices and other resources. It acts as a bridge between the user and the resources of the system.
- **Resource Management:** It is the duty of a Kernel to share the resources between various process in such a way that there is uniform access to the resources by every process.
- **Memory Management:** Every process needs some memory space. So, memory must be allocated and deallocated for its execution. All these memory management is done by a Kernel.
- **Device Management:** The peripheral devices connected in the system are used by the processes. So, the allocation of these devices is managed by the Kernel.

Kernel Mode and User Mode

There are certain instructions that need to be executed by **Kernel only**. So, the CPU executes these instructions in the Kernel Mode only. For example, memory management should be done in Kernel-Mode only. While in the User Mode, the CPU executes the processes that are given by the user in the User Space.

2.System Library

System libraries are special functions or programs using which application programs or system utilities accesses Kernel's features. These libraries implement most of the functionalities of the operating system and do not requires kernel module's code access rights.

3.System Utility

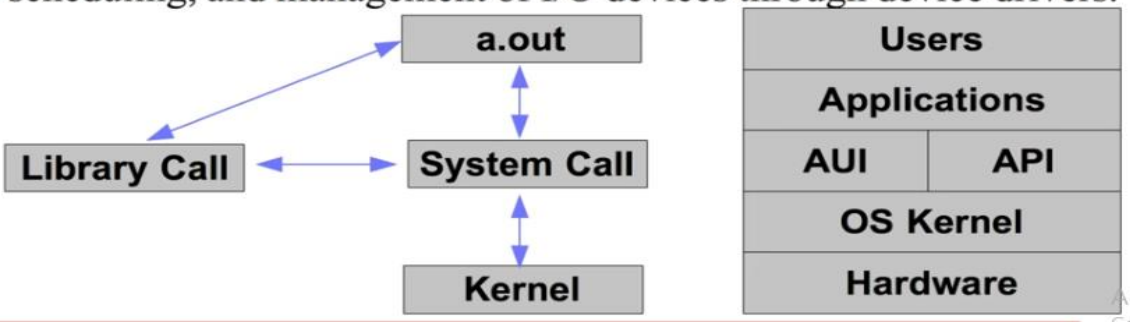
System Utility programs are responsible to do specialized, individual level tasks.

Introduction to systems programming and essential concepts

In computer there are two types of software

1. **Systems software**
2. **Application software**

- ***Application programming*** aims to produce software which provides services to the user, e.g., word processors, browsers, multi-media applications, etc.
- ***System Programming*** aims to produce software which provides services to computer hardware, e.g., disk defragmenter. The next step is writing kernel code to manage main memory, disk space management, cpu scheduling, and management of I/O devices through device drivers.



a.out – executable file access kernel services while at library call which farther calls system call to access kernel.

System Programmer Perspective

- A system programmer write programs that may have to acquire data
  - from a file(s) (residing on a local or remote disc) that may have been opened by some other user(s),
  - from other programs running on the same or different machine,
  - from the operating system itself.
- After processing, the programs may have to write results to a shared resource, which other processes are also writing, or results may need to be delivered to another process asynchronously, i.e., not when the process asked for it, but at some later unpredictable time. **The challenge is to master these concepts.**

Operating System Services

Some important tasks a kernel performs are:

- File Management
- Process management
- Memory management
- Information management
- Signal handling
- Synchronization
- Communication (IPC)
- Device management

Two methods by which a program can make requests for services from the Kernel:

- By making a system call (entry point built directly into the kernel)
- By calling a library routine that makes use of this system call

Activate Win

What is systems programming?

- **system programming- is the art of writing system software.**

System software lives at a low level, interfacing directly with the kernel and core system libraries.

Benefits of learning system programming

- **you get a better understanding of how the operating system works. this can ultimately help you in developing efficient programs.**
- **Cloud computing, high performance computing, distributed systems are some of the areas where system programming is heavily used.**
-

Systems call

system programming starts with system calls.

What is systems call?

- **A system call is a controlled entry point into the kernel, allowing a process to request that the kernel perform some action on the process's behalf. why?**
  It is not possible to directly link user-space applications with kernel space. For reasons of security and reliability, user-space applications must not be allowed to directly execute kernel code or manipulate kernel data

- **The kernel makes a range of services accessible to programs via the system call application programming interface (API). These services include, for example, creating** a new process, performing I/O, and creating a pipe for interprocess communication.
-

Why do we need systems calls in OS?

The two main reasons why such instructions are not available directly for the process.

- **(i) protection**

  The OS must do many checks to evaluate the right of the process to request such an instruction.

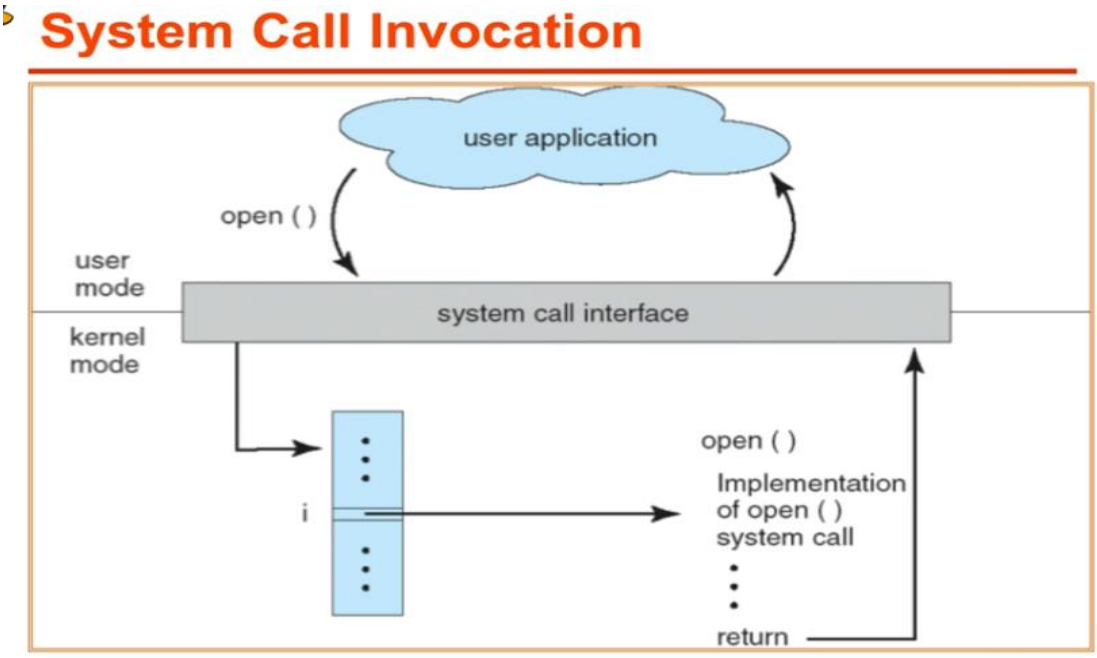
  (ii) update

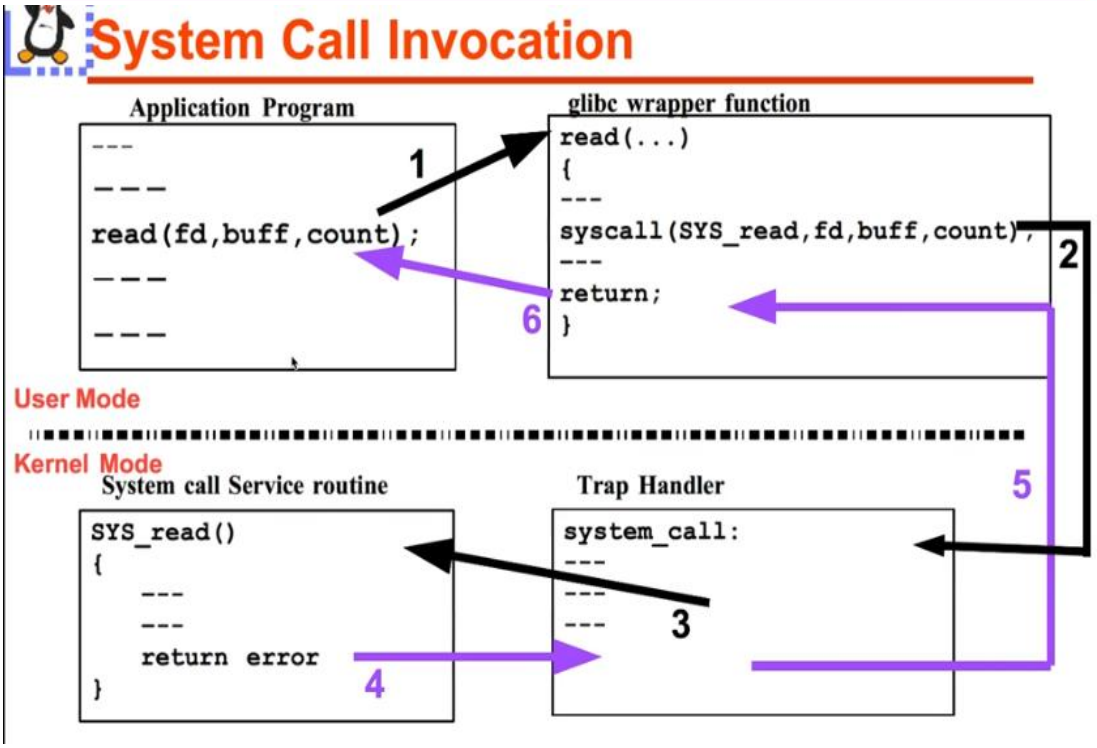
  The OS needs to update its data structures when the instruction is performed. If we take folk system call, the needs to update many data structures related to processes to take into account the new process, add it in the running queue, etc.

- **Reading and writing from files demand system calls.**
- **If a file a system wants to create or delete files, systems calls are required.**
- **System calls are used for creation and management of new processes.**
- **Network connections need systems for sending and receiving packets.**
- **Access to hardware devices like scanner, printer, need a system call.**

Before going into the details of how a system call works, we note some general points:

- A system calls changes the processor state from user mode to kernel mode, so that the CPU can access protected kernel memory.
- The set of system calls is fixed. Each system call is identified by a unique number. (This numbering scheme is not normally visible to programs, which identify system calls by name.)
- Each system call may have a set of arguments that specify information to be transferred from user space (i.e., the process's virtual address space) to kernel space and vice versa.





It is possible to invoke system calls directly by system call but since most program rely on glibe functions for system call.

- **From a programming point of view, invoking a system call looks much like calling a C function.**
  1. **The application program makes a system call by invoking a wrapper function in the C library**

2. **The wrapper function must make all of the system call arguments available to the system call trap-handling routine (described shortly). These arguments are passed to the wrapper via the stack, but the kernel expects them in specific registers. The wrapper function copies the arguments to these registers.**
3. **Since all system calls enter the kernel in the same way, the kernel needs some method of identifying the system call. To permit this, the wrapper function copies the system call number into a specific CPU register (%eax).**
4. **The wrapper function executes a trap machine instruction (int 0x80), which causes the processor to switch from user mode to kernel mode and execute code pointed to by location 0x80 (128 decimal) of the system's trap vector.**
5. **In response to the trap to location 0x80, the kernel invokes its system_call() routine (located in the assembler file arch/i386/entry.S) to handle the trap. This handler: a) Saves register values onto the kernel stack (Section 6.5). b) Checks the validity of the system call number. c) Invokes the appropriate system call service routine, which is found by using the system call number to index a table of all system call service routines (the kernel variable sys_call_table). If the system call service routine has any arguments, it first checks their validity; for example, it checks that addresses point to valid locations in user memory. Then the service System Programming Concepts routine performs the required task, which may involve modifying values at addresses specified in the given arguments and transferring data between user memory and kernel memory (e.g., in I/O operations). Finally, the service routine returns a result status to the system_call() routine. d) Restores register values from the kernel stack and places the system call return value on the stack. e) Returns to the wrapper function, simultaneously returning the processor to user mode.**
6. **If the return value of the system call service routine indicated an error, the wrapper function sets the global variable errno (see Section 3.4) using this value. The wrapper function then returns to the caller, providing an integer return value indicating the success or failure of the system call.**

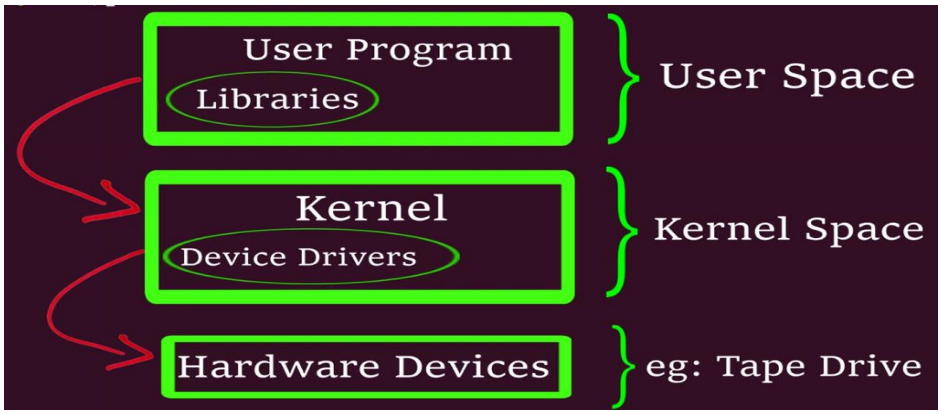
System calls (Read, write, open, close) *low level functions* program code illustration using Linux.

Open- opens a file /device

Read- reads from an open file

Write- write to a file/device

Close- close a file/ device



how to Access Tape drive from user space mode we need a system call via kernel.

Systems call program.

```c
#include<unistd.h>
#include<stdio.h>
#include<fcntl.h>

int main()
{
int fd;
char buffer[80];
char msg[50] = "hello ifocus institute";
fd = open ("check.txt", O_RDWR);
printf("fd = %d",fd);
if (fd != -1)
{
printf("\n check.txt opened with read write access\n");
write(fd, msg , sizeof(msg));
lseek(fd,0,SEEK_SET);
read(fd, buffer, sizeof(msg));
printf("\n %s was written to my file\n", buffer);
close (fd);
}

return 0;
}
```

Systems call functions highlighted.

Code explained

1. **#include<unistd.h> - name of the header file that provides access to the POSIX (***portable operating system interface is a family of standards specified by*

IEEE for maintaining compatibility between operating systems) **operating API.**
i.e close function is declared in unistd.h

The file contains the system call number that is passed to the kernel through the register(EAX) when a system call is invoked.

2. **#include<fcntl.h>- file control options**
   fcntl.h *function can be used to change the properties of a file that is already open.*

i.e. file descriptor flags.

3. line line 7: **int fd (file descriptor)- integer that uniquely identifies an open file of the process.**

Standard file descriptors – when any process starts FD 0,1,2 open automatically.

Read from stdin => read from fd 0: whenever we write any character from keyboard it read keyboard, it read from stdin through fd 0.

Write to stdout => write to fd1: whenever we see any output to the video screen, its from the file named /dev/tty and written to standard out in screen **through fd1.**

Write to stderr => write to fd2: we see any error to the video screen, it is also from that file write to stderr in the screen through fd2.

3. line 10 : fd =open(" check.txt",0_RDWR);

```
NAME
       open, openat, creat - open and possibly create a file

SYNOPSIS
       #include <sys/types.h>
       #include <sys/stat.h>
       #include <fcntl.h>

       int open(const char *pathname, int flags);
       int open(const char *pathname, int flags, mode_t mode);
```

Open system call function- has two parameters * pathname, int flags.

```
The argument flags must include one of the following access modes: O_RDONLY, O_WRONLY, or O_RDWR.  These
request opening the file read-only, write-only, or read/write, respectively.
```

```
 7 int fd;
 8 char buffer[80];
 9 char msg[50] = "hello ifocus institute";
10 fd = open ("check.txt", O_RDWR);
11 printf("fd = %d",fd);
12 if (fd != -1)
```

Line 12 : if(fd!= -1)

If the systems call fails, the it returns an error…but in our program its successful no error.

Line 15 : `write(fd, msg , sizeof(msg));`

```
SYNOPSIS
       #include <unistd.h>

       ssize_t write(int fd, const void *buf, size_t count);
```

Write function call has three parameters

Fd -

Buffer – where the read data is to be stored and the number of bytes to be read from the file. (array of bytes unlike string) buffers are low level and can be can refer to any data type.

Count – number of bytes and store them into the buffer.

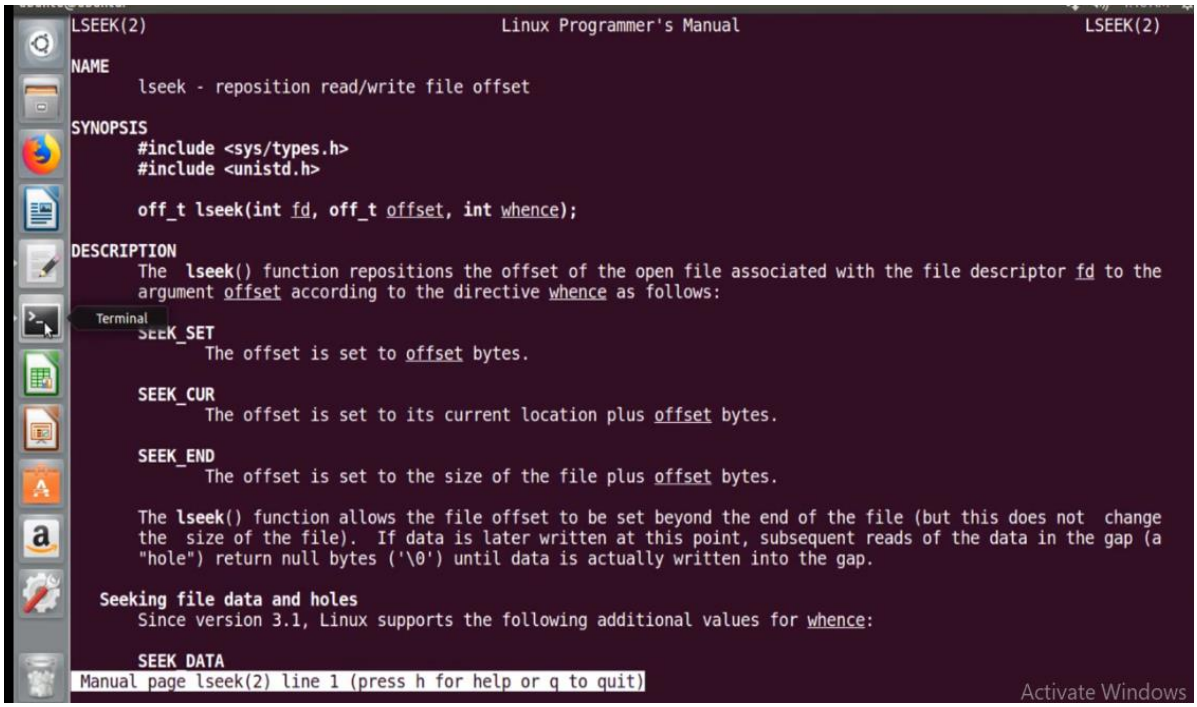
Line 16 : `lseek(fd,0,SEEK_SET);`

Lseek- used to change location of Read/Write pointer of a file descriptor

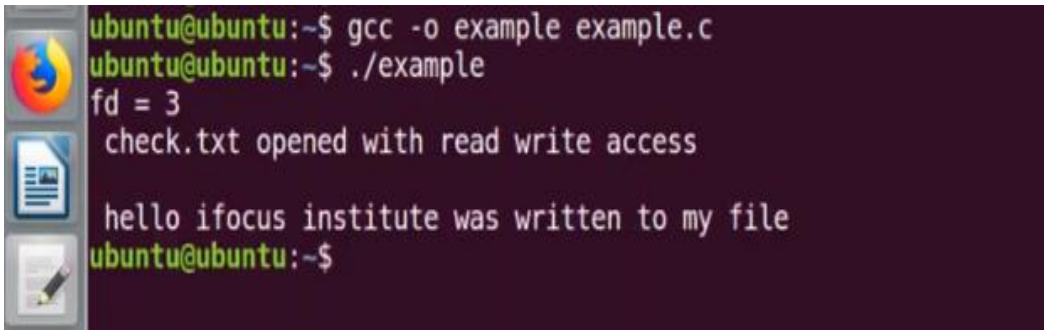


Just click on terminal type man followed by [linux command name]

Man means user linux manual **i.e man lseek.**



Output of the program

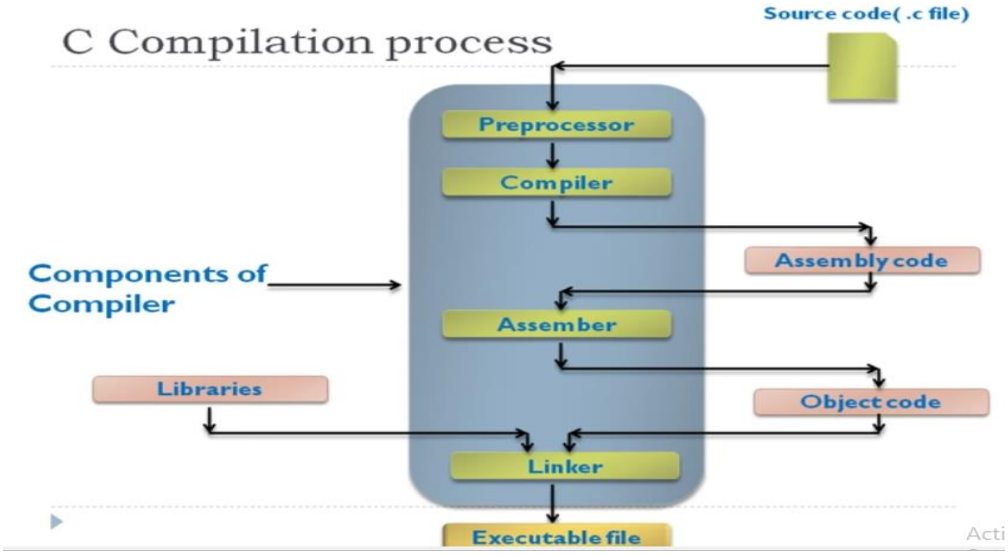


Compiling C files with Gcc compiler using Linux

Compilation process

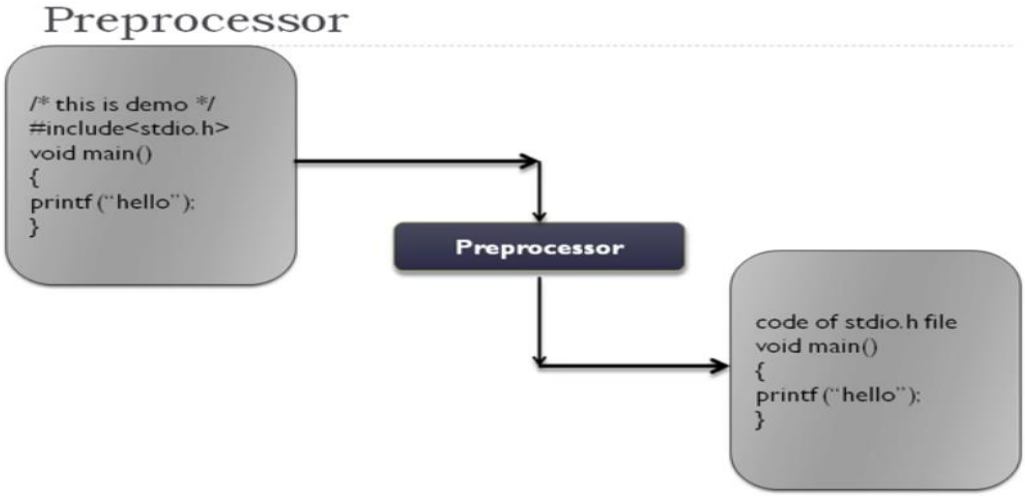
Four Steps of Compilation:

i. preprocessing,
ii. compiling,
iii. assembly,
iv. linking.

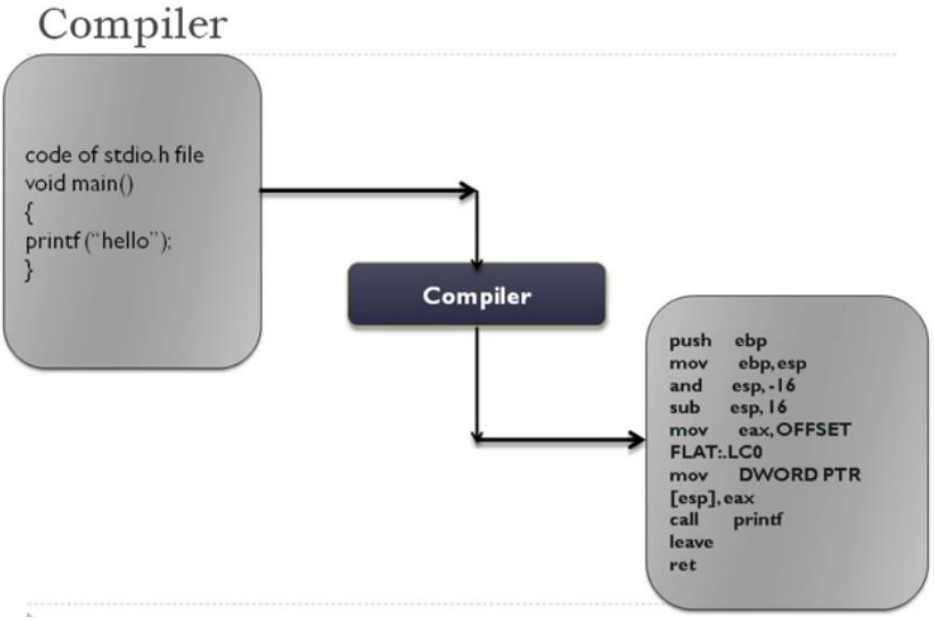


1. **Preprocessing**. The preprocessor obeys commands that begin with # (known as directives) by:

- removing comments
- expanding macros
- expanding included files



2. **compiling -** It takes the output of the preprocessor and generates assembly language, an intermediate human readable language, specific to the target procer

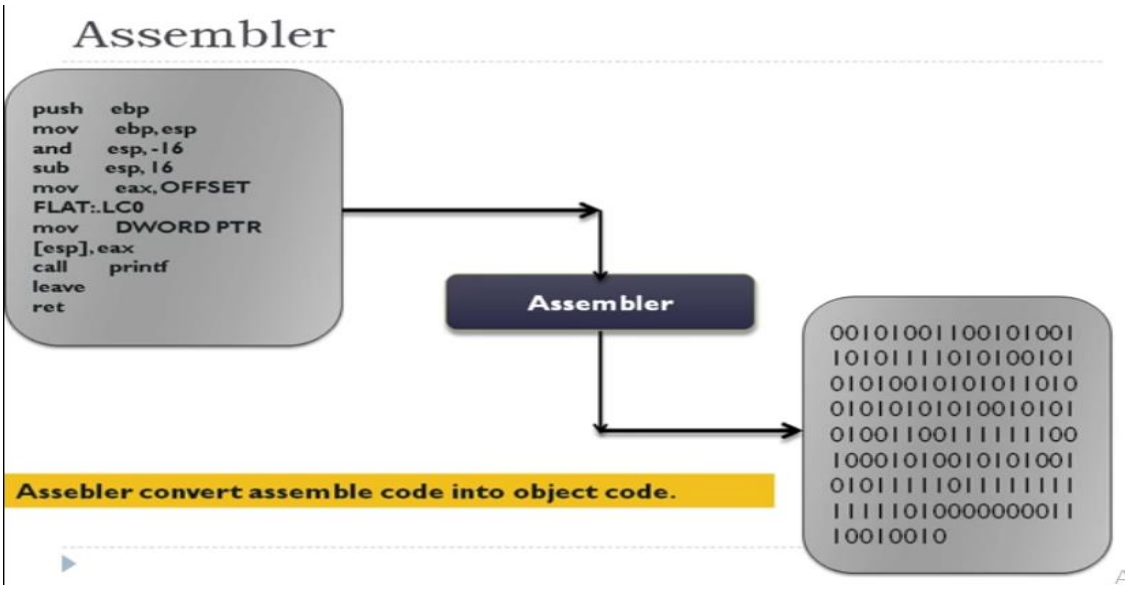


Why Assembly Language?

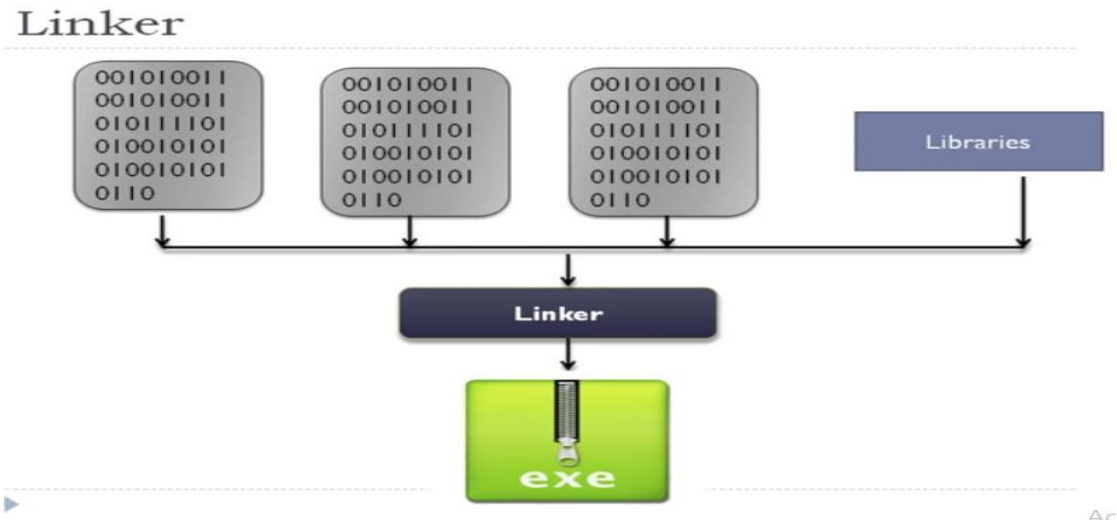
- By learning assembly language, the programmer can write the code to access registers and retrieve the memory address of pointers and values. It mainly helps in speed optimization that increases efficiency and performance.

- Assembly language learning helps in understanding the processor and memory functions. If the programmer is writing any program that needs to be a compiler, that means the programmer should have a complete understanding of the processor. Assembly language helps in understanding the work of processors and memory. It is cryptic and symbolic language.

3. **Assembler**



4. **Linker**



How to Install Kali Linux 2020.1b in VirtualBox on Windows 10
https://www.youtube.com/watch?v=V_PayI5FlgQ&t=5s

lesson 2: we will start with our system programming practical's using **LINUX OS and C programming language.**

using kali linux version because of Network security topic.

NB: any other LINUX version is okay

i. **Redhat**
ii. **Ubuntu**
iii. **Fedora etc ..but for class demos Kali Linux**

Systems programming lesson 2

COM 312: BSc Computer science/BSc IT

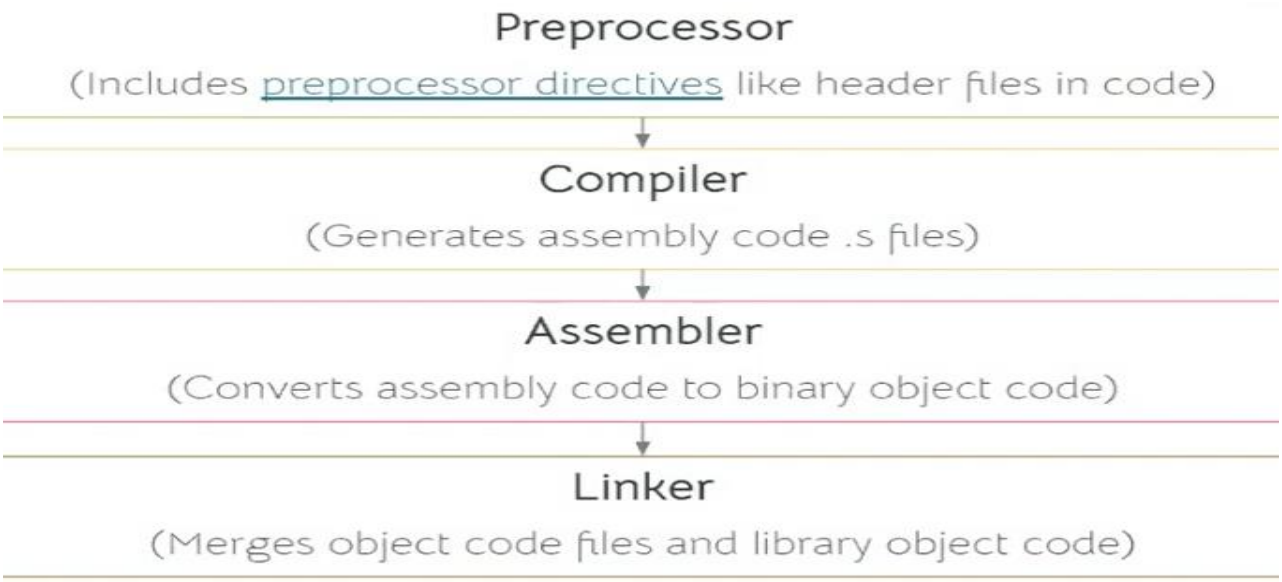
Topics area coverage.

- **Preparing your tool Box**
- **GNU gcc and gdb**
- **NASM (Netwide Assembler)**
- **Unix readlf, objdump, od, nm, idd and id config**
- **Creating your own static and dynamic libraries**

C code compilation process using GNU Gcc (compiler collection)

How is C code compiled?

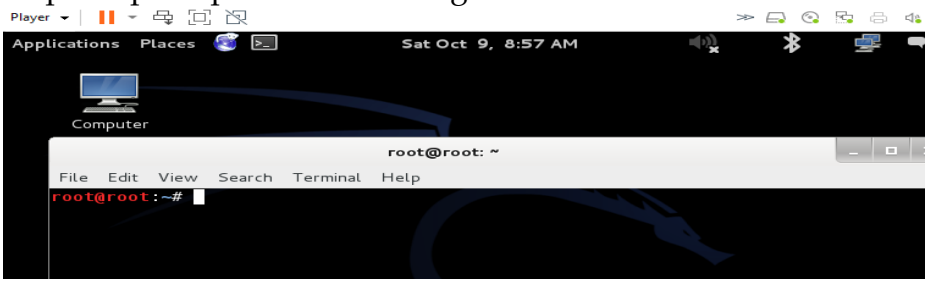
Four steps process.



Open your kali linux Tool

Click >terminal

Step 1. Open up a terminal using kali linux



Step 2. Use a text editor to create the C source code. (using nano editor)

Type the command

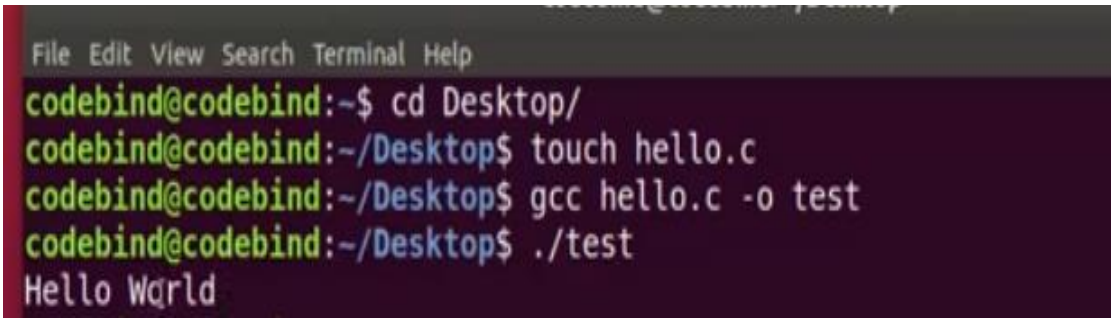
nano hello.c

and enter the C source code below:

```
#include<stdio.h>
Main ()
{ printf ("Hello World\n");
Return 0;
}
```

Close the editor window.

Or



Linux commands

cd- change directory

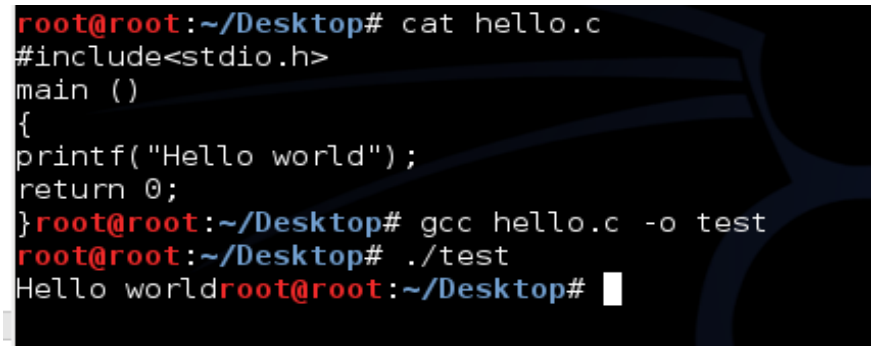
touch – the touch command is often used just to create an empty file.

Gcc- Gnu compiler collection.

-o – flag for output after compilation

./- forward slash command for running the test binary file.

Cat- concatenate file and print on the standard output.



Step 1 and 2 : Preprocessor to compiler(to generate assembly code.s files)

```
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ ls
myProgram.c
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ vi myProgram.c
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ gcc -S myProgram.c
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ ls
myProgram.c  myProgram.s
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ ls -l *.s
-rw-r--r-- 1 alohachurchill alohachurchill 657 Sep 16 23:57 myProgram.s
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ cat myProgram.s | less
```

Unix commands

Ls- list directory contents (list information about FILES current directory)

Vi myprogram.c – vim editor enables viewing of program code

Gcc-s myprogram.c – to make assembler code(low level programming) from our C File.

Ls- converting myprogram.c to my program.s File

Ls –l *.s – searches for all assembly Files

Cat myprogram.s |less- opposite of more (to view assembly hello world code)

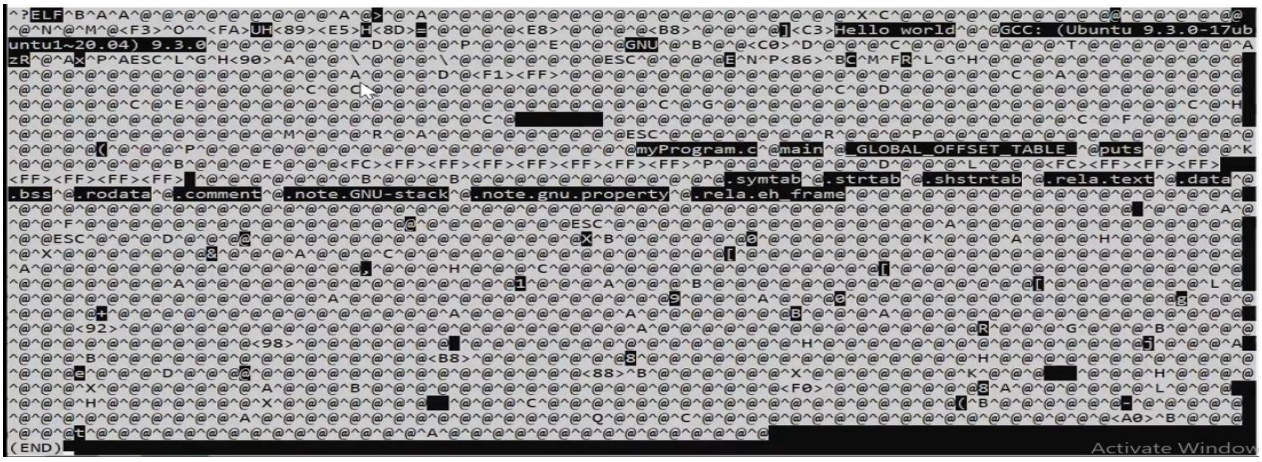
```
.LC0:
        .string "Hello world"
        .text
        .globl  main
        .type   main, @function
main:
.LFB0:
        .cfi_startproc
        endbr64
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq    %rsp, %rbp
        .cfi_def_cfa_register 6
        leaq    .LC0(%rip), %rdi
        call    puts@PLT
        movl    $0, %eax
        popq    %rbp
        .cfi_def_cfa 7, 8
        ret
        .cfi_endproc
.LFE0:
        .size   main, .-main
        .ident  "GCC: (Ubuntu 9.3.0-17ubuntu1~20.04) 9.3.0"
        .section        .note.GNU-stack,"",@progbits
        .section        .note.gnu.property,"a"
        .align 8
        .long    1f - 0f
        .long    4f - 1f
:_
```

Step 3 – Assembler

Converts assembly code to machine object code

```
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ ls
myProgram.c
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ vi myProgram.c
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ gcc -S myProgram.c
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ ls
myProgram.c  myProgram.s
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ ls -l *.s
-rw-r--r-- 1 alohachurchill alohachurchill 657 Sep 16 23:57 myProgram.s
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ cat myProgram.s | less
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ gcc -c myProgram.s
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ ls
myProgram.c  myProgram.o  myProgram.s
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ cat myProgram.o | less
```

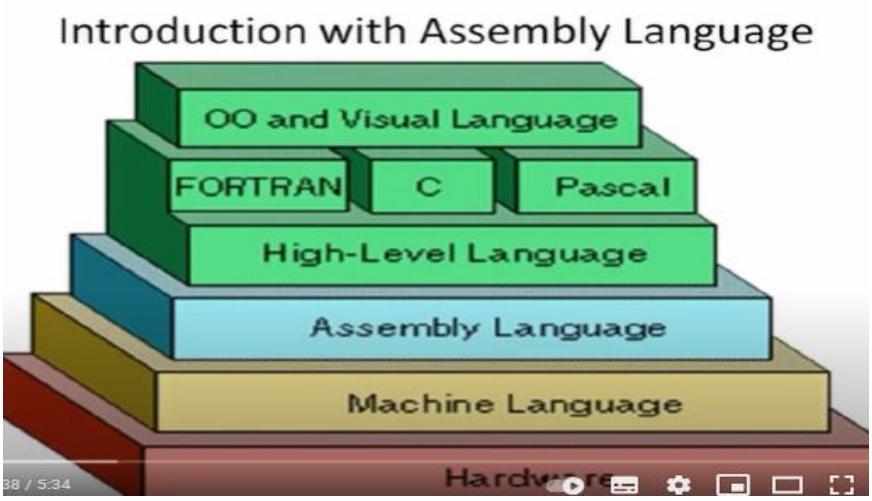
Object code output



Last step 4: linker – merges object code files and libraly object code

```
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ ls
myProgram.c
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ vi myProgram.c
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ gcc -S myProgram.c
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ ls
myProgram.c  myProgram.s
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ ls -l *.s
-rw-r--r-- 1 alohachurchill alohachurchill 657 Sep 16 23:57 myProgram.s
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ cat myProgram.s | less
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ gcc -c myProgram.s
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ ls
myProgram.c  myProgram.o  myProgram.s
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ cat myProgram.o | less
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ gcc myProgram.c -o myProgram
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ ls
myProgram  myProgram.c  myProgram.o  myProgram.s
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ ./myProgram
Hello world
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ cat myProgram | less
```

NASM (netwide Assembler)



- What is assembly language and its importance?
- What you need to know to start writing your first assembly program.

What is assembly language and its importance?

- Processor –Dependent
- Low level programming language (Human Readable machine language)
- Assembly code is converted into executable machine code by a utility program referred to as an assembler
  - Each family of **CPUs have its own set of instructions** or Assembly language for Handling Various operations such as
    i. Getting input from keyboard
    ii. Performing various task
    iii. Displaying information on the screen

There are main different types of Assembly languages for different types of CPU Architecture







ARM – Architecture used on phones among others (i.e ios, Android)

MIPS- Architecture used in video games console among others (i.e sony PlayStation)

X86 – Architecture used on intel PCs and other (16 bit to 64bit)

Why Assembly?

- Direct communication with the hardware
- Writing device driver
- Access to specialized processor instructions
- Operating systems design
- Embedded systems programs
  i. e A good example is Aviation industry uses a lot of assembler like computer Assembler Engineer

learning Assembly will help us understand

- How data is represented in memory and other external devices
- How programs interface with your operating system, processor, and BIOS
- How the processor accesses and executes instructions?
- How instructions access and process data
- How a program accesses external devices?

Basic features of a computer Hardware

- Registers
- Memory
- Processor

Understanding Registers

- One of the main tools to write programs in X86 assembly are the **processor registers.**
- You can, informally, think of **registers** as variables that are built in the processor
- Using **registers** instead of memory to **store values**, makes the process faster and more efficient.

An assembly program can be divided into three sections −

- The **data section**, - declaring initialized data or constants. This data does not change at runtime. You can declare various constant values, file names, or buffer size, etc
- **The bss section**, - The **bss** section is used for declaring variables

- The **text** section. - The **text** section is used for keeping the actual code. This section must begin with the declaration **global _start**, which tells the kernel where the program execution begins.

  syntax

```
section.text
   global _start
_start:
```

The Hello World Program in Assembly

```
section .text
   global _start     ;must be declared for linker (ld)

_start:              ;tells linker entry point
   mov  edx,len      ;message length
   mov  ecx,msg      ;message to write
   mov  ebx,1        ;file descriptor (stdout)
   mov  eax,4        ;system call number (sys_write)
   int  0x80         ;call kernel

   mov  eax,1        ;system call number (sys_exit)
   int  0x80         ;call kernel

section .data
msg db 'Hello, world!', 0xa  ;string to be printed
len equ $ - msg      ;length of the string
```

Output

```
Hello, world!
```

Code explanation

Mnemonic in assembly language

Mnemonic language – memory aid **to represent machine operation, or opcode**

MOV- move (assign) one value to another

ADD- Add two values

SUB- subtract one value from another

MUL-multiplying two values

JMP- jump to a new location

CALL-call a procedure

EAX- to store the return value of a function and as a special register for certain calculations**.** (**general purpose register**)

EBX-base pointer to the data section

ECX-counter for string and loop operations

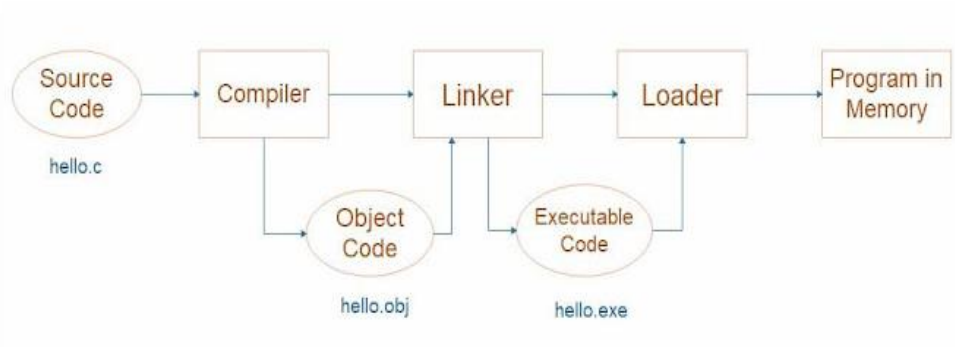
EDX- 1/0 pointer

Unix readlf, objdump, od, nm, idd and id config

Unix read elf- Executable Linking Format. This is the file which is loaded by the Kernel to create a process.

NB: A relocatable file **holds code and data** suitable for linking with other object files to create an executable

- ELF is the file format used for object files (`.o`'s), binaries, shared libraries and core dumps in Linux.
- ELF is used as standard file format for object files on Linux. Prior to this, the a.out file format was being used as a standard but lately ELF took over the charge as a standard.



- An object file is **a computer file containing object code**, that is, machine code output of an assembler or compiler. The object code is usually **relocatable**, and not usually directly executable.
- Since then, the ELF format is in use by several different operating systems. Among others, this includes Linux, Solaris/Illumos, Free-, Net- and OpenBSD,

QNX, BeOS/Haiku, and Fuchsia OS [2]. Furthermore, you will find it on mobile devices running Android, Maemo or Meego OS/Sailfish OS as well as on game consoles like the PlayStation Portable, Dreamcast, and Wii.

ELF supports:

- Different processors (cross plat-form)
- Different data encoding
- Different classes of machines

What is executable?

Binary file with instruction to be performed by the CPU

Why use ELF Files?

- Generic understanding of how an operating system works
- Development of software
- Digital Forensics and Incident Response (DFIR)
- Malware research (binary analysis)

Revision Questions

1. Why does Assembly It require less memory and execution time?
2. Why does Assembly language allow hardware-specific complex jobs in an easier way?

Systems programming lesson 3

COM 312: BSc Computer science

Topics area coverage.

UNIX File System

- Linux Directory structures
- Basic shell scripting in Linux
- Unix and Posix file system
- Inodes, API, UNIX kennel support for files

Linux Directory structures

What's a computer File system? - A file system is the *methods and data structures* that an operating system **uses to keep track of files on a disk or partition**; that is, the way the files are organized on the disk.

- The word is also used to refer to a partition or disk that is used to store the files or the type of the file system.

3 types of File types

i. **Ordinary files –** *Data, text, or program instructions.*
ii. **Special files –** *files provide access to hardware such as hard drives, CD-ROM drives, modems, and Ethernet adapters.*
iii. **Directories –** *store both special and ordinary files.*

Linux directory structures

- Linux keeps all the devices and data in form of files in different locations.
- It contains a root directory > sub-directories > files.
  NB: in Linux system you won't find drive D, E. etc only /slash or /mount point.

Unix uses a hierarchical file system structure, *much like an upside-down tree, with root (/) at the base of the file system and all other directories spreading from there*.

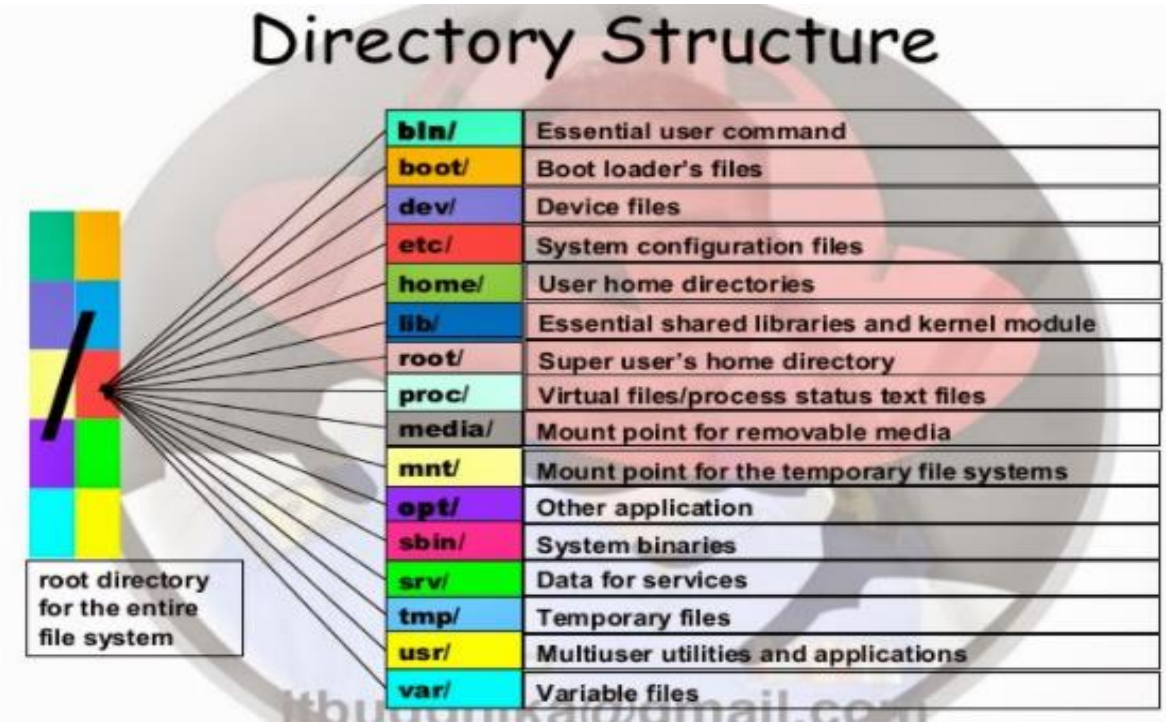
A Unix **file system** is a collection of files and directories that has the following properties

- It has a root directory (/) that contains other files and directories.
- Each file or directory is uniquely identified by its name, the directory in which it resides, and a unique identifier, typically called an **inode**.
- By convention, the root directory has an **inode** number of **2** and the **lost+found** directory has an **inode** number of **3**. **Inode numbers 0 and 1 are**

not used. File inode numbers can be seen by specifying the **-i option** to **ls command**.

- It is self-contained. There are no dependencies between one filesystem and another.

the Linux file system structures and understand the meaning of individual high-level directories.



/bin : All the executable binary programs (file) required during booting, repairing, files required to run into single-user-mode, and other important, basic commands viz., cat, du, df, tar, rpm, wc, history, etc.
/boot : Holds important files during boot-up process, including Linux Kernel.
/dev : Contains device files for all the hardware devices on the machine e.g., cdrom, cpu, etc
/etc : Contains Application's configuration files, startup, shutdown, start, stop script for every individual program.
/home : Home directory of the users. Every time a new user is created, a directory in the name of user is created within home directory which contains other directories like Desktop, Downloads, Documents, etc.
/lib : The Lib directory contains kernel modules and shared library images required to boot the system and run commands in root file system.
/lost+found : This Directory is installed during installation of Linux, useful for recovering files which may be broken due to unexpected shut-down.

/media : Temporary mount directory is created for removable devices viz., media/cdrom.
/mnt : Temporary mount directory for mounting file system.
/opt : Optional is abbreviated as opt. Contains third party application software. Viz., Java, etc.
/proc : A virtual and pseudo file-system which contains information about running process with a particular Process-id aka pid.
/root : This is the home directory of root user and should never be confused with '/'
/run : This directory is the only clean solution for early-runtime-dir problem.
/sbin : Contains binary executable programs, required by System Administrator, for Maintenance. Viz., iptables, fdisk, ifconfig, swapon, reboot, etc.
/srv : Service is abbreviated as 'srv'. This directory contains server specific and service related files.
/sys : Modern Linux distributions include a /sys directory as a virtual filesystem, which stores and allows modification of the devices connected to the system.
/tmp :System's Temporary Directory, Accessible by users and root. Stores temporary files for user and system, till next boot.
/usr : Contains executable binaries, documentation, source code, libraries for second level program.
/var : Stands for variable. The contents of this file is expected to grow. This directory contains log, lock, spool, mail and temp files.

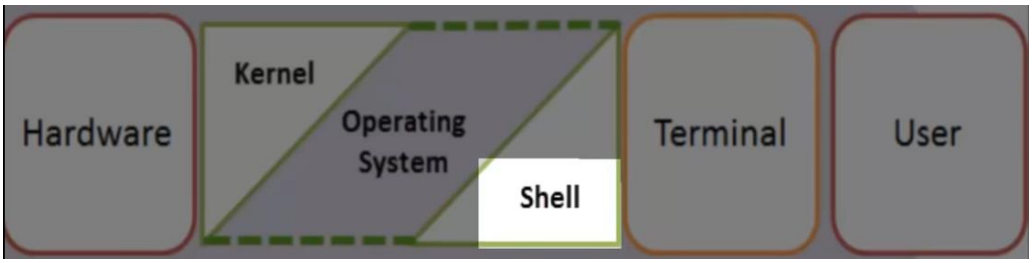
Basic shell scripting in Linux

Why should I learn Shell Programming at all?

One reasons to learn shell programming. the major reason is systems administration. **It is a powerful programming method that can help you learn the command-line better,** *save time*, and do away with tedious file management tasks.

used for multiple purposes

a) such as automating repetitive tasks
b) backup and maintenance of databases,
c) monitoring a system
d) creating a programming environment,
e) linking existing programs together, and various other tasks.



- Shell is a program which interprets user commands through **CLI** like Terminal

Shell vs Terminal

- **Shell >** is a program that is responsible for the execution of an instruction and returning the output.
- **Terminal >** is responsible to send instructions to the shell by taking input from the user and displaying the out of the instruction to the user.

Linux Shell

- Linux has a kernel which talks to the hardware. But a kernel does not understand the language of a user.
  And this is where a shell comes into the picture.
- A shell is an interpreter between the kernel and the user.
- It gathers input from the users and executes the programs,

There are various types of shell present which are listed below –

1. Bourne shell (sh)/Bash - Bash is a Unix shell and command language written by Brian Fox for the GNU Project as a free software replacement for the Bourne shell. Its name is an acronym for **Bourne-again shell**. (Kali Linux Debian-version uses Bash shell)
2. Korn shell (ksh)
3. Bourne again shell (bash)
4. POSIX shell (sh)
5. C shell (csh)
6. TENEX/TOPS C shell (tcsh)

Command to view different shells.

```
root@root:~# which shell
root@root:~# cat /etc/shells
# /etc/shells: valid login shells
/bin/sh
/bin/dash
/bin/bash
/bin/rbash
/usr/bin/tmux
/usr/bin/screen
root@root:~#
```

What is the shell scripting?

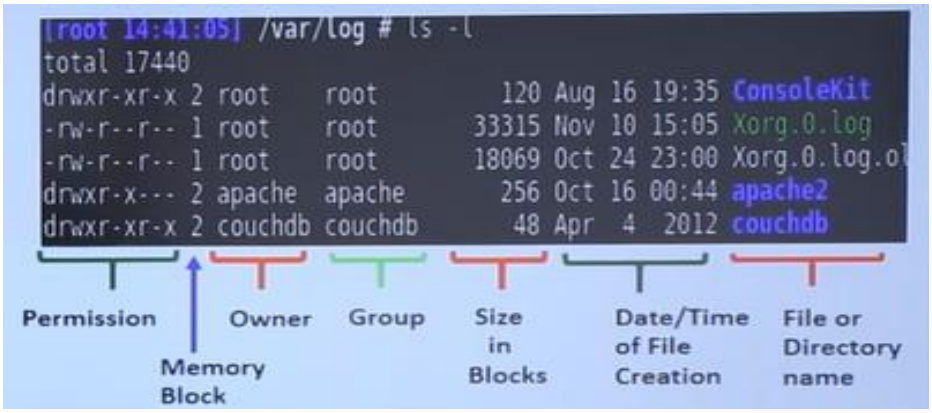
- Shell scripting is a group commands written in a sequence to automate

  Your :

  Regular boring stuff or Repetitive work.

- **Basic Shell Scripting Commands in Linux**: cat, more, less, head, tail, mkdir, cp, mv, rm, touch, grep, sort, wc, ls, cut and, more.

  Let's pick ls-l (list long) shell scripting command.



Why bash shell?

bash shell is an advanced shell.

- **bash** is an sh- compatible shell that incorporates useful features from the korn shell(ksh) and C shell (Csh)
- Now a day bash shell is the default shell for most of the UNIX/Like systems. used for multiple purposes Example of Bash shell scripting is used to automate tasks.
  a) such as automating repetitive tasks

- ✓ install and configure httpd on RHEL & Ubuntu
- ✓ install jenkins on Tomcat servers on RHEL & Ubuntu
- ✓ find all servers FQDN and Uptime and export it to CSV file.
- ✓ Install and configure Ansible setup on AWS cloud .(Ansible Controller and Required numbers numbers of Node)

How to Automate these Tasks?

i. Step 1: get all manual steps (commands)
ii. Step2: write a shell script
iii. Step3: now execute shell script.

The idea behind the statement above demonstrates ….installation of may be 100 servers using cloud computing. to save time system admin should write bash command using TOM cat ..to Automate the task from manual installation

shell scripting > introduction $ she bang

So what's the difference between the **shell** $ **bash** interpreter *command in Linux programming?*

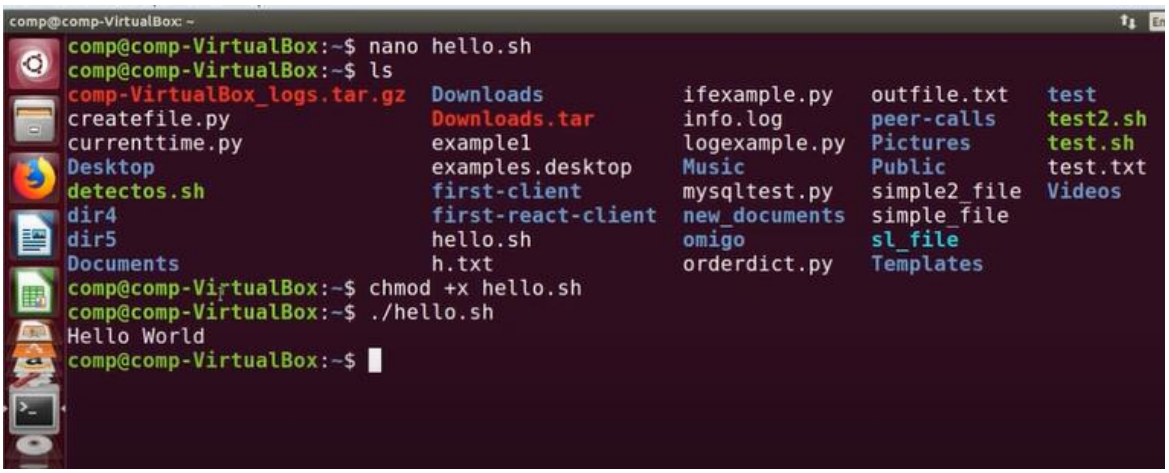
Bash interpreter > *offers streamlined experience. (offers full functionalities of the Unix/Linux support*)

Shell interpreter > *does not support full functionalities of the Unix/Linux support*)

How to write hello world bash shell script?

The procedure is as follows:

1. Create a new file called hello.sh using a text editor such as nano or vi:
   ```
   nano hello.sh
   ```
2. Add the following code:
   ```
   #!/bin/bash
   echo "Hello World"
   ```
3. Set the script executable permission by running chmod command:
   ```
   chmod +x hello.sh
   ```
4. Run or execute the script using following syntax:
   ```
   ./hello.sh
   ```
   OR
   ```
   sh hello.sh
   ```
   OR
   ```
   bash hello.sh
   ```



Code explanation

- First line of the script (`#!/bin/bash`) is shebang. It tells the Linux/Unix how to run the script.
- **Chmode + x :** change mode

The +x parameter is **used to add the x permission** which is the symbol for the execute permission.

- The execution permission will give the execution ability to the owner user or group of the file.
  - *i.* The + *will add provided ability* and **-** *minus will remove the provided ability.*

Bash script with user prompt and input validation

You can accept input from the keyboard and assign an input value to a user defined shell variable using read command.

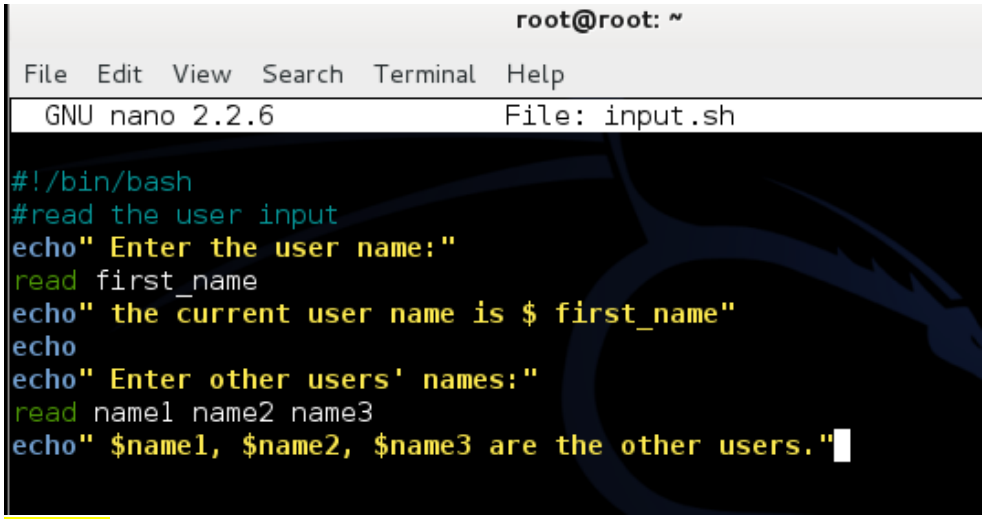
Option	Meaning
-a ANAME	The words are assigned to sequential indexes of the array variable ANAME, starting at 0. All elements are removed from ANAME before the assignment. Other NAME arguments are ignored.
-d DELIM	The first character of DELIM is used to terminate the input line, rather than newline.
-e	**readline** is used to obtain the line.
-n NCHARS	**read** returns after reading NCHARS characters rather than waiting for a complete line of input.
-p PROMPT	Display PROMPT, without a trailing newline, before attempting to read any input. The prompt is displayed only if input is coming from a terminal.
-r	If this option is given, backslash does not act as an escape character. The backslash is considered to be part of the line. In particular, a backslash-newline pair may not be used as a line continuation.

Option	Meaning
-s	Silent mode. If input is coming from a terminal, characters are not echoed. invisible input from user i.e. take password or secret data.
-t TIMEOUT	Cause **read** to time out and return failure if a complete line of input is not read within TIMEOUT seconds. This option has no effect if **read** is not reading input from the terminal or from a pipe.
-u FD	Read input from file descriptor FD.

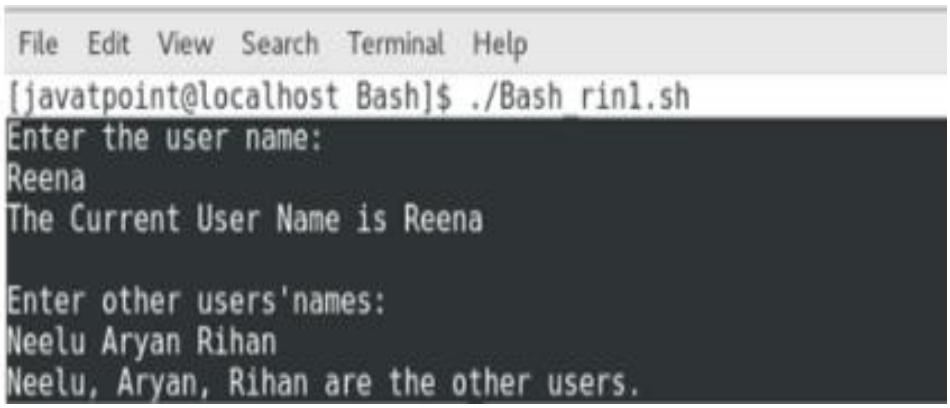
i. Bash script with user prompt"

Example 1:

In this example, we read both the single and multiple variables from the Bash Script by using read command.



output



Example – using simple read command

Create a bash file with the following script to know how to use the read command with a variable. After running the script, the program will wait for the user input. When the user types the data and press enter, the data will be stored in the answer variable. The value of the answer variable will be printed later.

```
#!/bin/bash
echo -n "What is your favorite food: "
# Assign input value into a variable
read answer
# Print the value of the variable
echo "Oh! you like $answer!"
```

Exercise

Run the script and give your output!!!!

Unix and Posix file system

There are many popular operating systems, including Windows, MacOS, Android, Chrome OS… **If each OS has its own unique standard and API**, *it would be hard to distribute the same software to multiple OS platforms.* Therefore, we need a universal standard that works across multiple OS, which is called POSIX.

The C API

POSIX **defines its standards in terms of the C language**. Therefore, **programs are portable to other operating systems at the source code level**. Nonetheless, we can also implement it in any standardized language.

The POSIX C API adds more functions on top of the ANSI C Standard for a number of aspects:

- File operations
- Processes, threads, shared memory, and scheduling parameters
- Networking

- Memory management
- Regular expressions

I nodes, API, UNIX kennel support for files

Linux and other Unix-like Operating systems **maintain consistency by treating everything as a file** (even the hardware devices). *The keyboard, mouse, printers, monitor, hard disk, processes, even the directories are treated as files in Linux*. The regular files contain data such as text (text files), music, videos (multimedia files), etc.

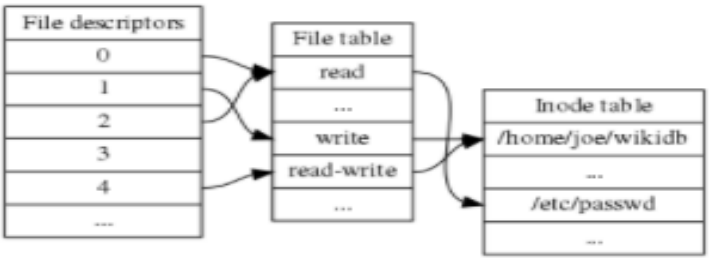
What is an inode in Linux? (index node)

(1) An inode is a *thing* (a data structure), not a number

(2) An "inode number" is the name/address of an inode.

Every Linux file or directory has an inode, and this inode **contains all of the file's metadata.**

- **I nodes store information about** files and directories **(folders),** *such as file ownership, access mode (read, write, execute permissions),* **and file type**.



File descriptors, file table and inode table in Unix[7]

- **NB**: For better performance make your directories smaller by sorting files into subdirectories rather having one large directory.

Why learn and use inode?

Inode is a Linux and other Unix-like data structure used to keep information about a file on your server. *This includes everything on your account including files, folders, emails, code,* etc. The number of inodes indicates the number of files and folders you have. Therefore, the more files and folders you have on your server, the higher your inode usage will be.

- Keep in mind, **inode usage** differs from **disk space usage.** Disk space refers to the size of your files size (KB, MB, GB). However, inode refers to the number of files you have.
- As an example, let's suppose you are using 20GB / 30GB on your server. This will make you well within your disk space usage. However, if you have hundreds of thousands of very small files within that 20GB, the inode usage could end up negatively affecting the performance of your server.

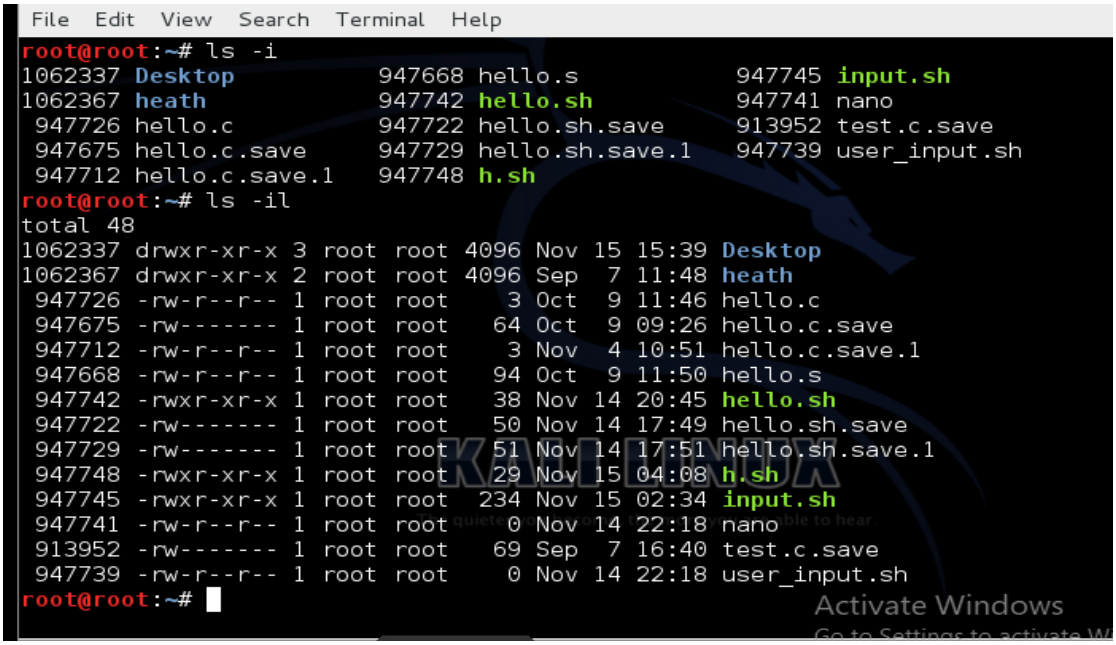
  I nodes

  • **Advantages**: – Fast access for small files (majority) – Supports large files – Supports sparse files

What's the difference between I node number and file descriptor?

i. **I node numbers** *are like pointers within the file system*, and **are stored within the file system itself.** (**In particular, a directory entry** is the combination of a name and an inode, and you can see this using ls -i.

- An inode number always refers to something on a device somewhere



The first number on the left represents the inode number of the file.

Within a POSIX system, a file has the following attribute contained in inode file

- Device ID (this identifies the device containing the file; that is, the scope of uniqueness of the serial number).
- File serial numbers.

- The file *mode* which determines the file type and how the file's owner, its group, and others can access the file.
- A link count telling how many hard links point to the inode.
- The User ID of the file's owner.
- The Group ID of the file.
- The device ID of the file if it is a device file.
- The size of the file in bytes.
- Timestamps telling when the inode itself was last modified (`ctime`, *inode change time*), the file content last modified (`mtime`, *modification time*), and last accessed (`atime`, *access time*).
- The preferred I/O block size.
- The number of blocks allocated to this file.

What can go wrong with inodes?

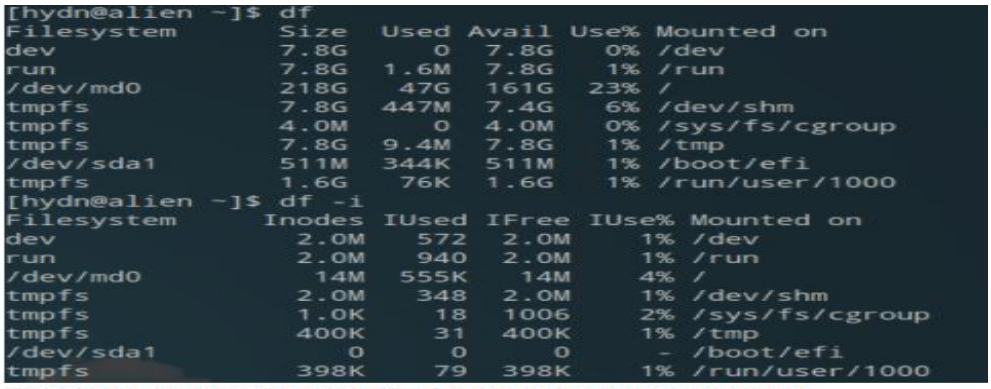
Running out of inodes can look weird, because it's likely you'll still have free disk space

The moment a file system runs out of inodes, all new files and folders will be rejected. Until that point, no adverse effects will manifest. Once inodes are 100% used, you'll begin to notice:

- Data loss
- Applications crashing
- OS restarting
- Processes don't restart
- Periodic tasks not firing

If you notice any of the above, use **df-i** to confirm that you've reached inode capacity

The **df command** (short for ***disk free***) is used to show the amount of free disk space available on Linux and other Unix-like systems and to understand the filesystems that have been mounted.



df with no arguments and df -i (show free inodes) output.

Revision Questions

1. Write a systems program using **bash scripting** that will prompt user input while saving the input into three variables. The read statement has the keyword "-a" to take multiple values from the user and save them to the one variable array "**NAMES**". At the last echo statement, all the inputted values are printed as array members within the string text. **[5 marks]**

2. Discuss various types of files supported by Unix. Also explain the use of chmod command. **[5 marks]**

3. Write a Shell script to check whether the given 3 number is prime or not. [3 marks]

Systems programming lesson 4

COM312: BSc Computer science

Topics area coverage.

Process management and scheduling
• Process identifications • Process trees, chains, and fans • Zombies and ophan processes • Process groups, sessions and controlling terminals • Unix daemons • Systems vR3 and systemVR4 scheduler

Process management and scheduling

- The most fundamental concept in modern OS is the **process.**
- The principle function of the OS is to **create, manage,** and **terminate processes**.
- To perform its process management functions, the OS maintains a description of each process, or process image.
- During its lifetime, a process moves among a number of states. The most of important of these are **Ready, Running, and Blocked**

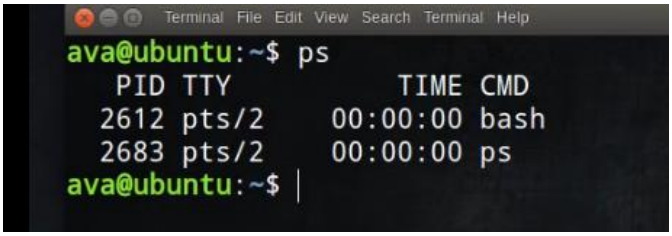
Operations on Processes

The processes in the systemexecute concurrently and they must be **created** and **deleted** dynamically thus the operating system must provide the mechanism for the creation and deletion of processes.

Program vs. Process

Definition: a **process** is the OS *representation of a program during its execution*

How we check the processes. using the ps command



Program vs process

Lets look at the difference between process and program

Process	Program
The process is basically an instance of the computer program that is being executed.	A Program is basically a collection of instructions that mainly performs a specific task when executed by the computer.
A process has a **shorter lifetime**.	A Program has a **longer lifetime**.
A Process requires resources such as memory, CPU, Input-Output devices.	A Program is stored by hard-disk and does not require any resources.
A process has a dynamic instance of code and data	A Program has static code and static data.
Basically, a process is the **running instance** of the code.	On the other hand, the program is the **executable code**.

- Why?
  - A user program is something static: *it is just a sequence of bytes stored in a "disk"*

- When user asks the O.S. for executing a program, in a multiprogrammed-multiuser environment, the system needs to represent that particular "execution of X" with some attributes
  - Which regions of physical memory is using
  - Which files is accessing
  - Which user is executing it
  - What time it is started
  - How many CPU time it has consumed?

- Every process in a UNIX system has the following attributes:

- **User program** the program to be executed
- **some user data** – the modifiable part of the user space. May include program data, user stack area, and programs that may be modified
- **a stack** – each stack has one or more last-in-first-out (**LIFO**) stacks associated with it. A stack is used to store parameters and calling addresses for procedure and system calls.

- **Process control Block-** Data needed by the OS to control the process
- a unique process ID number (PID)

process states

- As a process executes, it changes state.
- The state of a process is defined in part by the current activity of that process.

Each process may be in one of the following states;

i. **NEW**–the process is being created
ii. **RUNNING**–instructions are being executed
iii. **WAITING**- the process is waiting for some event to occur (such as an I/O completion or reception of a signal)
iv. **READY**- the process is waiting to be assigned to a processor
v. **TERMINATED**- the process has finished execution

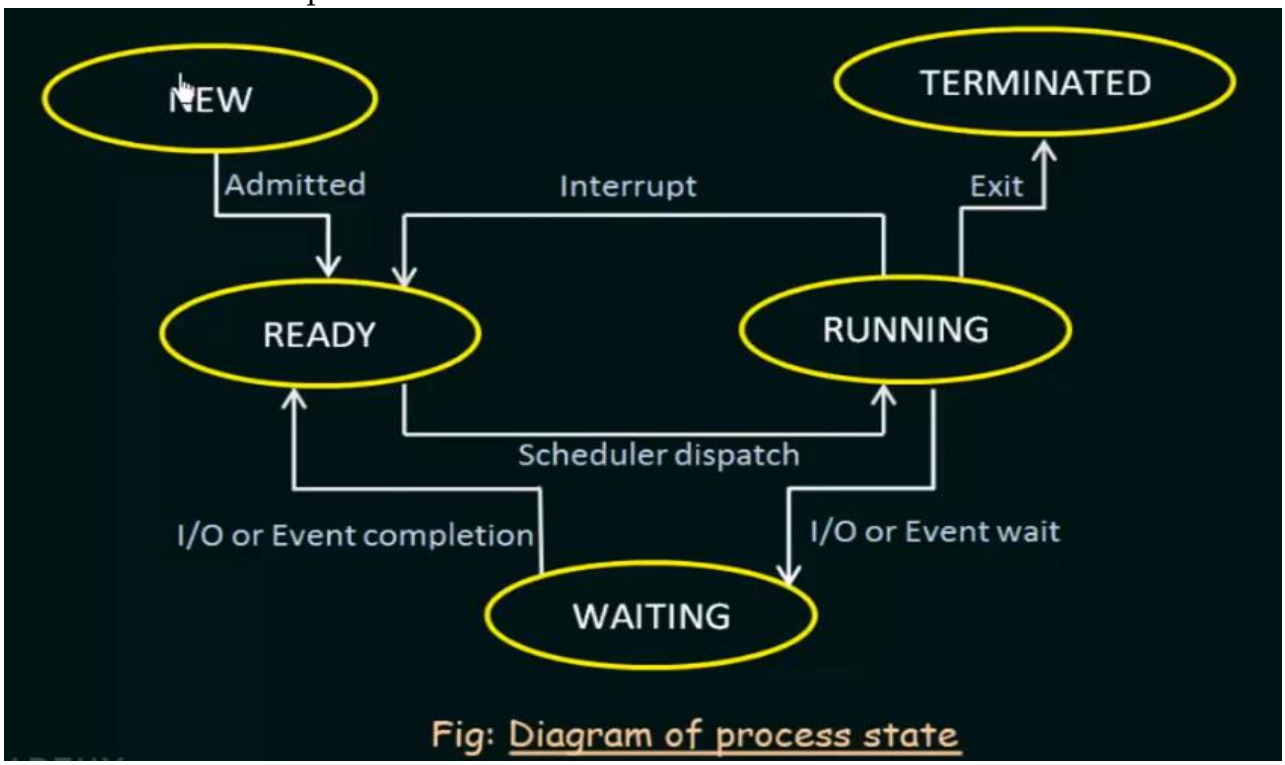


Fig: Diagram of process state

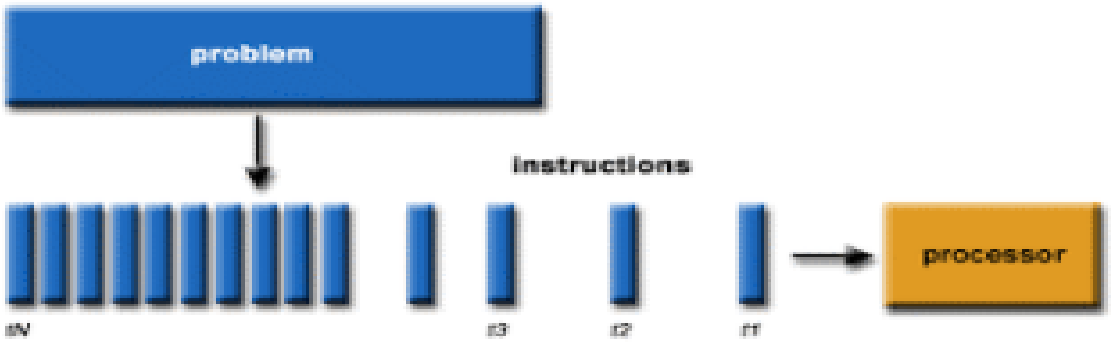
The Basic process creation using fork(), exec() wait(), exit() used in system call in Linux processes

FORK() – creates a new process
EXEC() – replaces current process image with a new process image
WAIT() – used parent process to wait for child's completion
EXIT() – used to terminate a process.

FORK & EXEC in LINUX

The main aim of fork() is for development of parallel computing

- **Parallel computing** refers to the process of executing several processors **an application or computation simultaneously.** Generally, it is a kind of computing architecture where the large problems break into independent, smaller, usually similar parts that can be processed in one go.



What is the purpose of folk?

- System call fork() is **used to create processes**.
- It takes no arguments and returns a process ID.
- The purpose of fork() is to create a new process, which becomes the child process of the caller. After a new child process is created, both processes will execute the next instruction following the fork() system call

Why does fork() return twice?

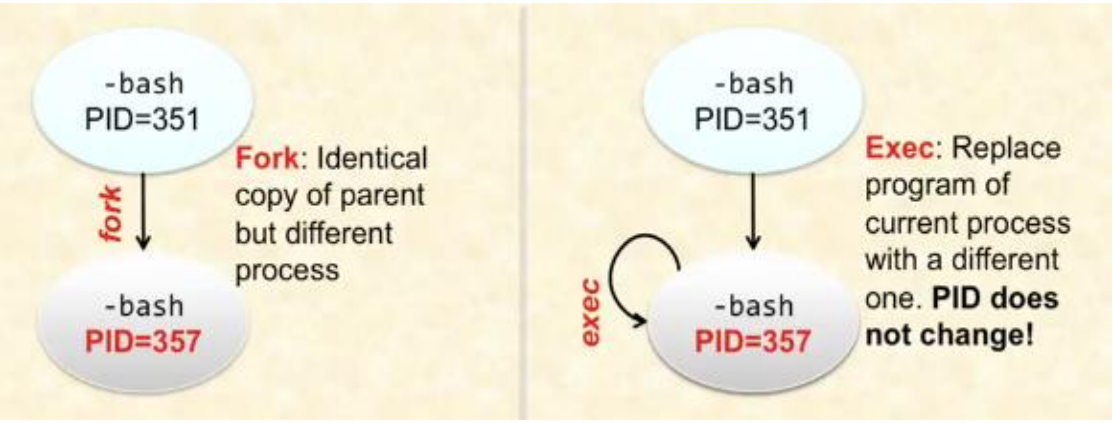
Once within the parent process and once within the child ... **The OS arranges for the fork() to return in the parent process with the pid of the child**, and for it to return in the child process with zero (**if things go well**). That is why they say that fork() returns twice.

- **Running a new program on Linux involves 2 steps.**

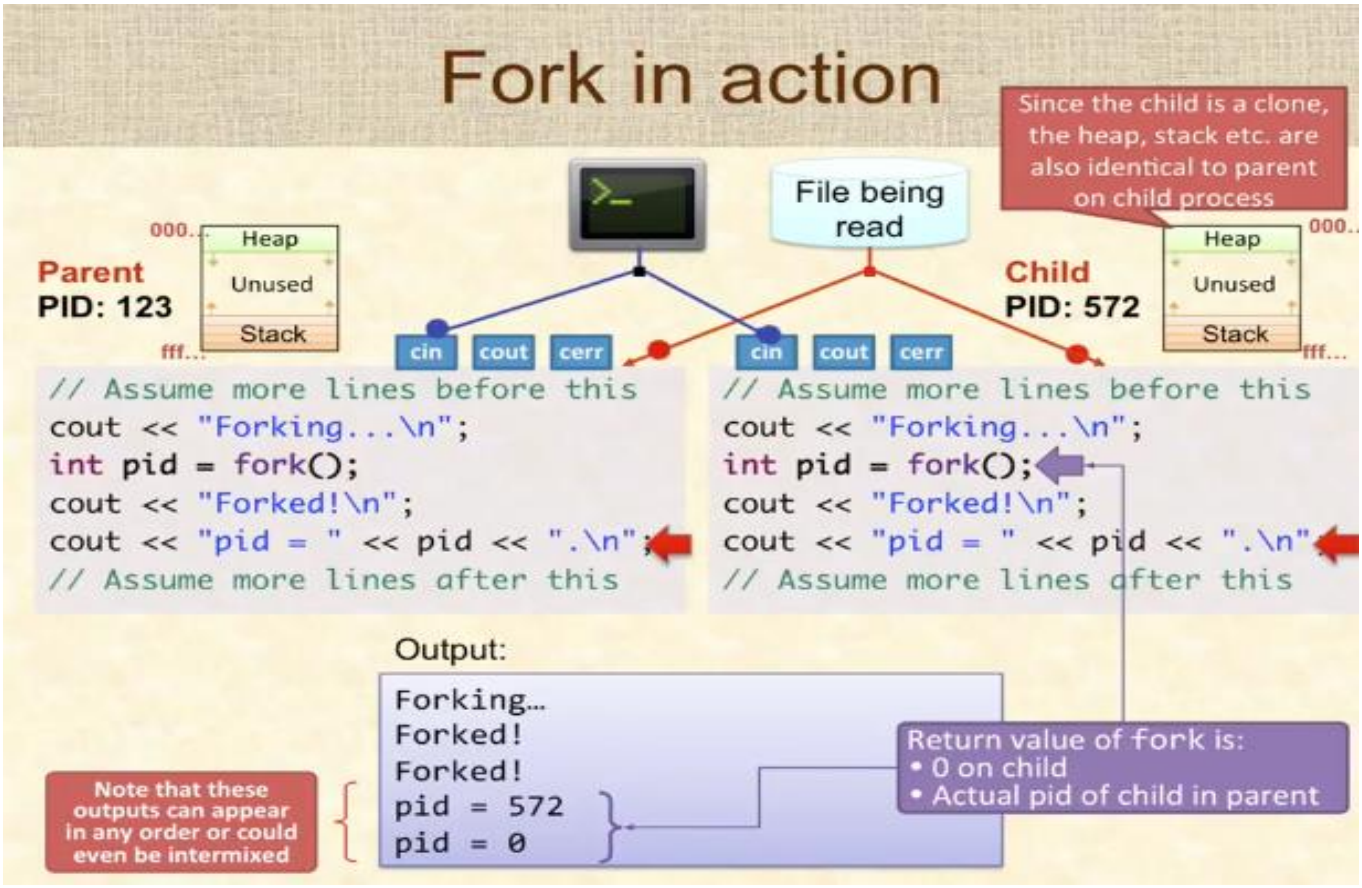
1. **FORK**: Clone current process
   - Fork() system call is used to create a separate, duplicate process.

- The fork() system call **allows one process, the parent, to create a new process, the child**. This is done by making the new child process an (almost) exact duplicate of the parent: the child obtains copies of the parent's *stack, data, heap, and text segments.* The term fork derives from the fact that we can envisage the parent process as dividing to yield two copies of itself

2. **EXEC**: Replace current program with a different one.
   - When an exec() system call is invoked, the program specified in the parameter to exec() will replace the entire process- including all threads.

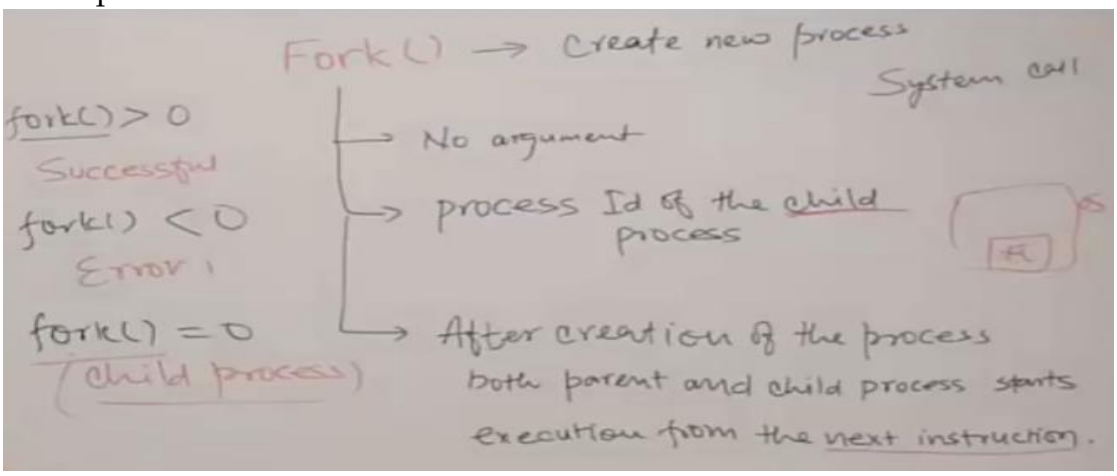


- **Fork is a system call used to clone a running process**
  - The cloned process is a child of the parent.
- **The parent and child are identical processes**
  - Have exactly the same stack (sequence of function calls)
  - Have same virtual memory (so pointers and other data is preserved)
  - Have same set of files and consequently have
    - Same set of open files, devices, sockets, etc.
- **The cloned child differs from the parent in:**
  - The process ID (pid)
  - The returned value from fork in the parent and child a slightly different
    - This is used to tell the difference between **parent** and **child** process



Code explained

- The program is running in memory
- Reads data from more files
- Calls fork() system call and creates new program identical copy to that of the parent.



Example Question

After a program executes the following series of fork() calls, how many new processes will result (assuming that none of the calls fails)?

fork();

fork();

fork();

Result

The first fork() call creates one new child. Both parent and child carry on to execute the second fork(), and thus each creates a further process, making a total of four processes. All four processes carry on to execute the next fork(), each creating a further child. Consequently, a total of seven new processes are created.

Forking processes has advantages

- **Threads do not enjoy many of these advantages**

1. **Fault tolerance**: if child process crashes parent process is unaffected heavily used in web-servers and databases to improve resilience
2. **Security**: parent and child can have different security settings (or even run as different users) for better auditing,
3. **Low latency**: since memory and I/O streams are identical start-up overheads are minimized
4. **Concurrency & performance**: parent and child can collaborate to perform different operations to improve performance
5. **Resource utilization**: processes get full set of resources enabling effective utilization of computational platforms
   - Easier to migrate processes across cloud infrastructures
   -

Zombies and orphan processes

What is Linux zombie process?

Zombie processes in Linux are sometimes also referred to as defunct or dead processes. They're **processes that have completed their execution**, but their entries are not removed from the process table. hasn't been properly cleaned up.

- A zombie is created by programs that crash or weren't adequately coded to handle the cleanup of a crash or exit.

- Technically speaking, when a process dies, the process descriptor remains in memory and becomes labeled EXIT_ZOMBIE. At this point, the parent process is notified that the child process has died with the SIGCHLD signal and the parent is supposed to execute the wait system call, which allows the parent to collect information from the now-dead process. This generally happens almost immediately. But not always.

Issue the command ps aux | grep 'Z and if any zombie processes are hanging around, you should see them. Those zombie processes will be listed with an associated Process ID.

To kill a zombie process, you can send the SIGCHLD signal with the command kill -s SIGCHLD PID(Where PID is the process ID of the zombie process).

Typically, you should don't see any zombie processes on your system. But if your system is behaving oddly (not as in trying to chomp your brains), check for zombie processes and, if you find any, kill them immediately.

Example illustrating Zombie process

Zombie step 1 .....we start from fork() process

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
      int p;
      p=fork();
      if(p>0)
       {
        printf("It's a parent process:%d",p);
       }
      else
       {
       printf("Child Process:%d",p);
       }
      return 0;
```

Code Explained

- First we examine the fork system call where by the parent will return positive number **while** child if successful will return zero value.

Zombie step 2 ..how to remove zombie from process table by use of wait () function

```c
#include<stdio.h>
#include<stdlib.h>
int main()
 {
     int p;
     p=fork();
     if(p>0)
      {
       printf("It's a parent process:%d",p);
       wait();
      }
     else
      {
      printf("Child Process:%d",p);
      }
     return 0;
 }
```

Code explanation

Using wait () function

- **Child process** will report to **parent process, then** that process will be removed from **the process table**

Zombie step 3 ... how to create a zombie process?

```c
#include<stdio.h>
#include<stdlib.h>
int main()
 {
     int p;
     p=fork();
     if(p>0)
      {
       printf("It's a parent process:%d",p);
       sleep(60);
      }
     else
      {
      printf("Child Process:%d",p);
      }
     return 0;
 }
```

Code explanation

Using sleep () function

- As long as **wait () function** is not called child process will terminate and will remain in the process table in that duration **Known as Zombie process**

Command to print all the process

```
ubuntu:~$ ps -e -o pid,stat,cmd
```

Executed program to illustrate Zombie process …

```
jgyan@ubuntu:~/Desktop$ gcc com.c
jgyan@ubuntu:~/Desktop$ ./a.out
Child Process:0^C
jgyan@ubuntu:~/Desktop$ gcc com.c
jgyan@ubuntu:~/Desktop$ ./a.out
Child Process:0
```

```
2849 Sl   /usr/lib/gvfs/gvfsd-metadata
2893 Sl   update-notifier
2900 Sl   gedit /home/jgyan/Desktop/com.c
2928 SNl  /usr/bin/python3 /usr/bin/update-manager --no-update
2953 Sl   /usr/lib/x86_64-linux-gnu/deja-dup/deja-dup-monitor
2964 Sl   gnome-terminal
2975 S    gnome-pty-helper
2976 Ss   bash
3040 S    [kworker/u128:1]
3069 S    /sbin/dhclient -d -sf /usr/lib/NetworkManager/nm-dhc
3427 Ss   /usr/sbin/cupsd -f
3430 S    /usr/lib/cups/notifier/dbus dbus://
3479 Sl   /usr/lib/x86_64-linux-gnu/unity-scope-home/unity-sco
3493 Sl   /usr/bin/unity-scope-loader applications/application
3495 Sl   /usr/lib/x86_64-linux-gnu/unity-lens-files/unity-fil
3518 Sl   /usr/lib/x86_64-linux-gnu/unity-lens-music/unity-mus
3551 Ss   bash
3735 S    [kworker/u128:0]
3750 T    ./a.out
3796 S    [kworker/u128:2]
3802 S+   ./a.out
3803 Z+   [a.out] <defunct>
3804 R+   ps -e -o pid,stat,cmd
jgyan@ubuntu:~$
```

Code output

Z + a.out child process, process waiting to report to the parent. But parent hasn't called **wait () function**. That why child process is z+ (zombie)

If you delete sleep(60) function? What will happen?

```
3479 Sl   /usr/lib/x86_64-linux-gnu/unity-scope-home/unity-sco
3493 Sl   /usr/bin/unity-scope-loader applications/application
3495 Sl   /usr/lib/x86_64-linux-gnu/unity-lens-files/unity-fil
3518 Sl   /usr/lib/x86_64-linux-gnu/unity-lens-music/unity-mus
3551 Ss   bash
3735 S    [kworker/u128:0]
3750 T    ./a.out
3796 S    [kworker/u128:2]
3816 S+   ./a.out
3818 R+   ps -e -o pid,stat,cmd
jgyan@ubuntu:~$
```

code explanation S+ single process which is the parent.

Unix daemons

Daemon: the term comes from Geek mythology, where daemons were guardian spirits.

- **Daemon**: Is a process that runs in the background and performs a specified operation at predefined times or in response to certain events.
  - Def: 2 – is a process that runs in the Background and is independent of control from all terminals.
  - There are **20-50** daemon processes running in a Unix system silently.
    There are several ways to start a daemon
    1. The system initialization scripts (/etc/rc)
    2. The inetd superserver
    3. Cron daemon : daemon that executes commands at a scheduled time.
    4. The at command
    5. From user terminals

- **Typical daemon process includes**: print spoolers, e-mail handlers, network systems and other programs that perform administrative tasks for the operating systems.

> **NB**: Almost all daemons have names that end with the letter "**d**". For example, httpd the daemon that handles the Apache server, or sshd (secure shell daemon,) which handles SSH remote access connections. **Linux often start daemons at boot time. Shell scripts stored in /etc/init.d directory are used to start and stop**

Functions of daemons

i. Open network port (such as port 80) and respond to network requests.
ii. Monitor systems such as hard disk health or RAID array.

How to find PID of the process?

The PID of the process can be found using "ps -ef command.

```
root@hardening:~# ps -ef
UID          PID  PPID  C STIME TTY          TIME CMD
root           1     0  1 11:15 ?        00:00:01 /sbin/init
root           2     0  0 11:15 ?        00:00:00 [kthreadd]
root           3     2  0 11:15 ?        00:00:00 [ksoftirqd/0
root           4     2  0 11:15 ?        00:00:00 [kworker/0:0
root           5     2  0 11:15 ?        00:00:00 [kworker/0:0
root           6     2  0 11:15 ?        00:00:01 [kworker/u2:
root           7     2  0 11:15 ?        00:00:00 [rcu_sched]
root           8     2  0 11:15 ?        00:00:00 [rcu_bh]
root           9     2  0 11:15 ?        00:00:00 [migration/0
root          10     2  0 11:15 ?        00:00:00 [watchdog/0]
root          11     2  0 11:15 ?        00:00:00 [kdevtmpfs]
root          12     2  0 11:15 ?        00:00:00 [netns]
root          13     2  0 11:15 ?        00:00:00 [perf]
root          14     2  0 11:15 ?        00:00:00 [khungtaskd]
root          15     2  0 11:15 ?        00:00:00 [writeback]
root          16     2  0 11:15 ?        00:00:00 [ksmd]
```

Coding Rules

- Some basic rules to coding a daemon prevent unwanted interactions from happening.
- We state these rules and then show a function, *daemonize,* that implements them.

1. The first thing to do is call umask to set the file mode creation mask to 0.
2. Call fork and have the parent exit.
3. Call setsid to create a new session.
4. Change the current working directory to the root directory.
5. Unneeded file descriptors should be closed.
6. Some daemons open file descriptors 0, 1, and 2 to /dev/null so that any library routines that try to read from standard input or write to standard output or standard error will have no effect.

Daemon process program in Linux environment

```
#include <stdio.h>
#include <unistd.h>
#include <syslog.h>
#include <fcntl.h>
void daemonize(const char *cmd)
{
        int i, fd0, fd1, fd2;
        pid_t pid;
        umask(0);
        if ((pid = fork()) < 0) {
        printf("%s: can't fork", cmd);
        exit(1);}
        else if (pid != 0) /* parent */
        exit(0);
        setsid();
        if (chdir("/") < 0) {
                perror("chdir: can't
                change directory to /");
                exit(1);
        }

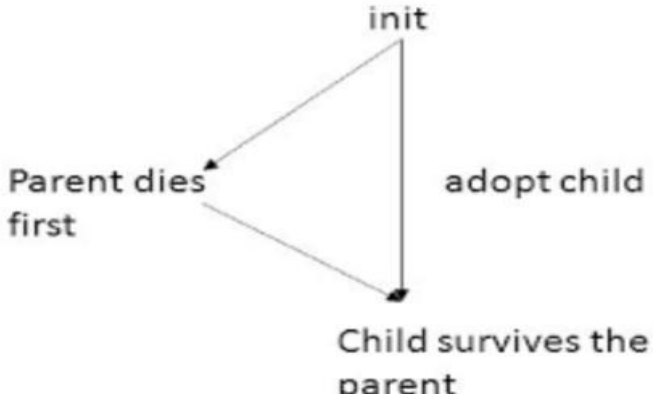
fd0 = open("/dev/null", O_RDWR);
fd1 = dup(0);
fd2 = dup(0);

exit(1);
}
```

Orphan process

- Created when a child process is running and the parent process is killed
- The system recognizes the orphaned child process.
- And passes the orphan process to init which then becomes the it's new parent process and terminates it.

- If a parent dies before its child process, the child is automatically adopted by the original init process.

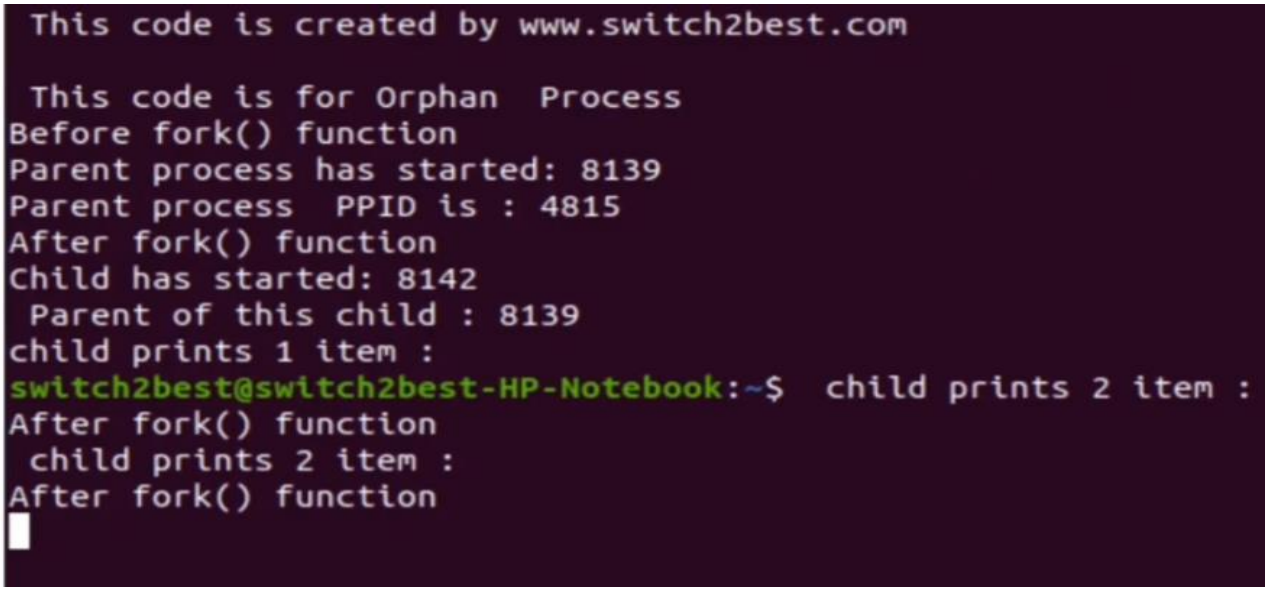


Write a C program to display the implementation of the orphan process

```
File  Edit  Format  View  Help
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/types.h>
#include<sys/stat.h>
#include<string.h>
printf("\n This code is created by www.switch2best.com\n");
printf("\n This code is for displaying orphan process\n");
int pid;
printf("Before fork () function\n");
pid=fork();
if(pid==0)
{
printf("child has started:%d\n",getpid ());
printf("parent of this child:%d \n",getppid());
printf("child print 1 item:\n");
sleep (30);
printf("child prints 2 item:\n");
}
else
{
printf("parent process has started:%d\n",getpid ());

printf("parent process PPID:%d\n",getppid ());
}
printf("After fork() function \n");
}
```

The output of orphan process



Revision Questions

1. What will be the output of the below C fork program assuming none of the fork fails.

```
#include<stdio.h>
#include<sys/types.h>
#include<unistd.h>
int main()
{
fork();
fork();
fork();
printf("hello\n");
return 0;
}
```

2. suppose that in the program the child process also needed to wait on the parent to complete some actions. What changes to the program would be required in order to enforce this?

Systems programming lesson 5 & 6 combined

COM 312: BSc Computer science

Topics area coverage.

UNIX inter process communication Tools

- Topic introduction
- Pipes and practices using P open () and P close () functions

UNIX Thread management and synchronization

- Posix standards and its sub group
- Threading Concept in systems programming
- Thread ID calls
- Mutex and semaphore in systems administration

UNIX inter process communication Tool

Defination1: - Inter process communication is the mechanism where process communicate with each other and with the kernel to coordinate their activities.to synchronize their action.

Definition 2: - it is a set of programming interface (API) which allows a programmer to coordinate activities among various program processes which can run concurrently in an operating system**. This allows a specific program to handle many user requests at the same time.**

- **Communication can be related or unrelated (**using pipes one process running in one terminal and another process in another terminal**) processes**
- **Linux supports a number of inter-process communication** (IPC)
  - Signals, shared memory and pipes are the most common
  - **But other IPCs are**
    -shared files
    -shared memory with (with semaphores)
    -message Queues
    -sockets

Inter-process communication Approaches

a) **Pipes**
b) **Message passing**
c) **Message queues**
d) **Direct communication**

e) **Indirect communication**
f) **Shared memory**
g) **FiFo**

Pipes

Pipes is widely used for communication between two related processes. This is a half-duplex method, so the first process communicates with the second process. However, in order to achieve a full-duplex, another pipe is needed.

Message passing

The process communicates with each other without resorting to shared variables.

Message Queues

A message queue is a linked list of messages stored within the kernel. It is identified by a message queue identifier. This method offers communication between single or multiple processes with full-duplex capacity.

Direct communication

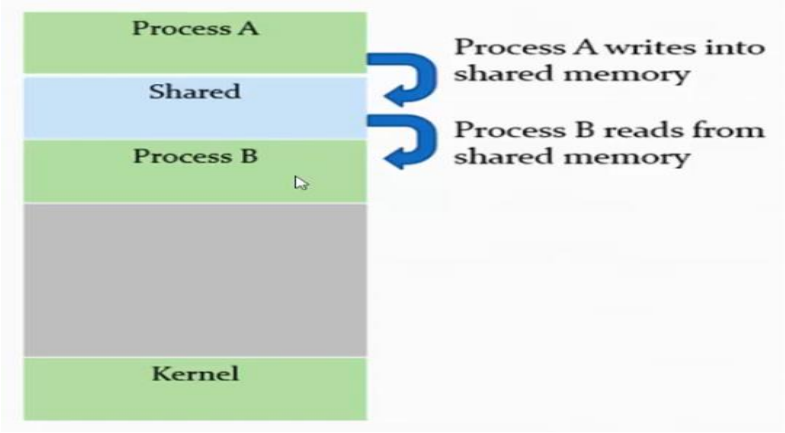
In this method, a link is established one pair of communicating process and between each pair, only one link exists.

What is Shared memory?

Shared memory is the fastest interposes communication mechanism. The operating systems maps a memory segment in the address space of several processes. **So that several processes can read and write in that memory segment without calling operating system** functions. However, we need some kind of between processes that read and write shared memory.

Why use processes use shared memory?

- Shared memory is memory that may be simultaneously accessed multiple programs with an intent to provide communication among them or avoid redundant copies.
- Shared memory run on single processor, parallel processor, or clustered processors



Shared memory is a concept where two or more process

Share resources from one memory address space.

Example:

Consider what happens when a server process wants to send an HTML file to a client process that resides in the same machine using network mechanisms.

- Single server that enables clients and servers to exchange data and instructions using main memory.
- Without shared memory, both client and server use use operating system buffers to accomplish the modification and exchange.
- With shared memory, the client writes its process directly into RAM and issues a semaphore value to flag sever attention. There is only 1 read and 1 write per communication, and the read/write is considerably faster than using system services. – to achieve higher performance computing

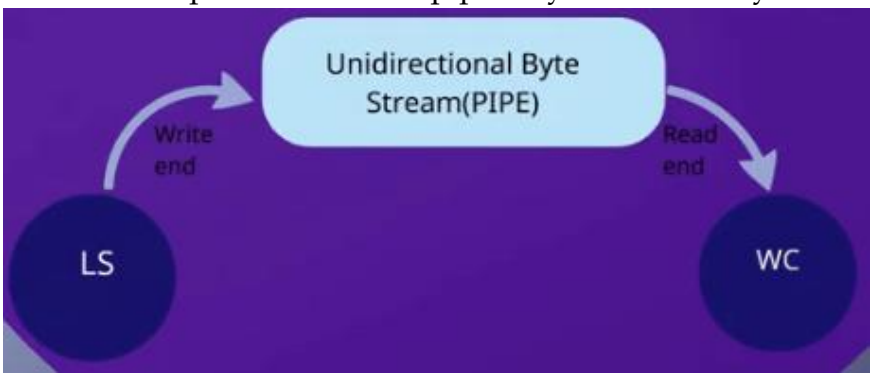
- **Need of inter process communication**
  i. Information sharing
  ii. Speedup
  iii. Modularity
  iv. Convenience

System V IPC is more heavy weight than BSD mmap and provides three methods of communication.

- Message Queues
- Semaphore and
- Shared segment.

Pipes and pipe demonstration using P open () and P close () functions

- Pipe is unidirectional (moving in a single direction) Byte stream. It has two ends means that two processes are connected to each other. (**connection between two processes**)
- Unlike other processes in **IPC** pipe only offer one-way communication.



LS– list the content of the directory

WC- word count the number of lines, bytes, words output from first command to input of second, and so forth

- Pipe () is a Linux system function.
- The pipe () system function is used to open **file descriptors** which are used to communicate between different Linux processes.
- Pipe is a special file that follows the FIFO mechanism

All about pipe () function:

The syntax of the pipe () function is:

int pipe (int pipefd [2]);

here, the pipe () function creates a unidirectional data channel for inter-process communication. You pass in an int(integer) type array pipefd consisting of 2 array element to the function pipe () then the pipe () function creates two file descriptors in the pipefd array.

The first element of the pipefd array, pipefd [0] is used for reading data from the pipe.

The second element of the pipefd array, pipefd [1] is used for writing data to the pipe.

On success, the pipe () function return 0. If an error occurs during pipe initialization, then pipe returns -1

Why do we use/Benefits of using pipes?

1. You can use them across different systems that may not exist at the same time. Suppose you want real-time communication of different machines. Just use FIFO files to share data.
2. As a file you can control ownership and permissions
3. You don't have to start the reading/ writing processes at the same time.

Example 1;

To create a new C source file pipe.c

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>

int main (void)
{
int pipefds[2];
if(pipe(pipefds)== -1)
{perror("pipe");
exit(EXIT_fAILURE);
}
printf("Read file Descriptor value:%d\n",pipefds[0]);
printf("write file Descriptor value:%d\n",pipefds[1];
return EXIT_SUCCESS;
}
```

Codes explained

Exit (EXIT_FAILURE)

If code fails the it exits there.

Popen and Pclose functions for inter-process communication.

Popen – opens a pipe stream to or from a process

- A common use of pipes is to send data to or receive data from a program being run in a sub process.
- **Popen** and **Pclose** functions ease this paradigm by eliminating the need to invoke pipe, fork, dup2, exec, and fd open.

Syntax

```
#include<stdio.h>
FILE  *popen(const char *command, const char *type);
```

Process for communication

send(or receive data

How to access the Linux manual

```
:~$ man open
```

```
POPEN(3)                    Linux Programmer's Manual                    POPEN(3)

NAME
       popen, pclose - pipe stream to or from a process

SYNOPSIS
       #include <stdio.h>

       FILE *popen(const char *command, const char *type);

       int pclose(FILE *stream);

   Feature Test Macro Requirements for glibc (see feature_test_macros(7)):

       popen(), pclose():
           _POSIX_C_SOURCE >= 2
               || /* Glibc versions <= 2.19: */ _BSD_SOURCE || _SVID_SOURCE

DESCRIPTION
       The  popen()  function  opens  a process by creating a pipe, forking, and invoking the shell.
       Since a pipe is by definition unidirectional, the type argument may specify only  reading  or
       writing, not both; the resulting stream is correspondingly read-only or write-only.

       The  command  argument  is  a  pointer to a null-terminated string containing a shell command
Manual page popen(3) line 1 (press h for help or q to quit)
```

Program to write into a pipe

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<string.h>
int main()
{
FILE *rd;
char buffer[50];
sprintf(buffer,"name first");
rd=popen("wc -c","w");
fwrite(buffer,sizeof(char),strlen(buffer),rd);
pclose(rd);
}
```

File* rd;

Data type file stream= file * rd the file name.

Char buffer [50] = buffer as a stream storage.

Sprintf () = print string to write data into the buffer

rd=popen ("wc –c", "w"); = rd name of the file that will use open pipe using write mode.

fwrite(buffer,sizeof(char),strlen(buffer),rd;

wc = cmd that reads either standard input or a list of computer files and generate one of the following statistics newline, word count, and byte count.

Example to illustrate the above written codes

```
 GNU nano 4.8                                    ipc1.c
#include<stdio.h>
#include<string.h>
int main()
{
        FILE *rd;
        char buffer[50];
        sprintf(buffer, "name first");
        rd=popen("wc -c", "w");
        fwrite(buffer,sizeof(char), strlen(buffer), rd);
        pclose(rd);
}
```

```
baljit@DESKTOP-MMRRHBE:~$ nano ipc1.c
baljit@DESKTOP-MMRRHBE:~$ gcc ipc1.c
baljit@DESKTOP-MMRRHBE:~$ ./a.out
10
baljit@DESKTOP-MMRRHBE:~$
```

UNIX Thread management and synchronization

Posix standards and its sub group

- **What is posix standards? "**Portability operating system interface**"** is the name of a family related standards specified by IEEE to define the application programming interface (API), along with shell utilities interface for the software compatible with variants of the UNIX operating system.

- Because many versions of UNIX exist today and each of them provides its own set of API functions, it is difficult for system developers to create applications that can be easily ported to different versions of UNIX.

- Some of **the subgroups** of POSIX are POSIX.1, POSIX.1b & POSIX.1c are concerned with the development of set of standards for system developers.

1. **POSIX. 1**
   - This committee proposes a standard for a base operating system API; this standard specifies APIs for the manipulating of files and processes.
   - It is formally known as IEEE standard 1003.1- 1990 and it was also adopted by the ISO as the international standard ISO/IEC 9945: 1: 1990.

2. **POSIX.1b**
   - This committee proposes a set of standard APIs for a real time OS interface; these include IPC (interposes communication)
   - This standard is formally known as IEEE standard 10034-1993.

3. **Posix.1c**
   - This standard specifies multi-threaded programming interface. This is the newest POSIX standard.

- These standards are proposed for a generic OS that is not necessary be UNIX system.
- E.g VMS from Digital Equipment corporation, OS/2 from IBM, & Windows NT from Microsoft corporation are POSIX-compliant, yet they are not UNIX systems.
- To ensure a user program conforms to POSIX.1 standard the user should either define the manifested constant_ POSIX_SOURCE at the beginning of each source module of the program (before inclusion of any header) as; # define_POSIX_SOURCE OR specify the –D_ POSIX_SOURCE option to a C++ compiler(CC) in a compilation; % CC –D_ POSIX_SOURCE *.C
- POSIX.1b defines different manifested constant to check conformance of user program to that standard. The new macro is POSIX_C_SOURCE and its value indicates POSIX version to which a user program conforms. Its value can be:

- **Program to check and display _POSIX_VERSION constant of the system on which it is run**

```
#define  _POSIX_SOURCE
#define  _POSIX_C_SOURCE   199309L
#include<iostream.h>
#include<unistd.h>
int main()
{
#ifdef _POSIX_VERSION
      cout<<"System conforms to POSIX"<<"_POSIX_VERSION<<endl;
#else
      cout<<"_POSIX_VERSION undefined\n";
#endif
      return 0;
}
```

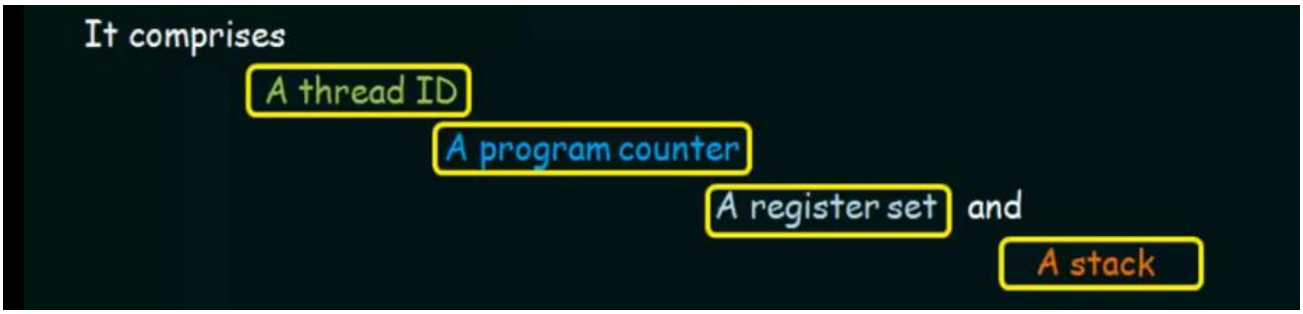
Threading Concept in systems programming

Knowing how to properly use threads should be part of every computer science and engineering student repertoire.

Threads?

Def 1: A thread is a basic unit of CPU utilization.

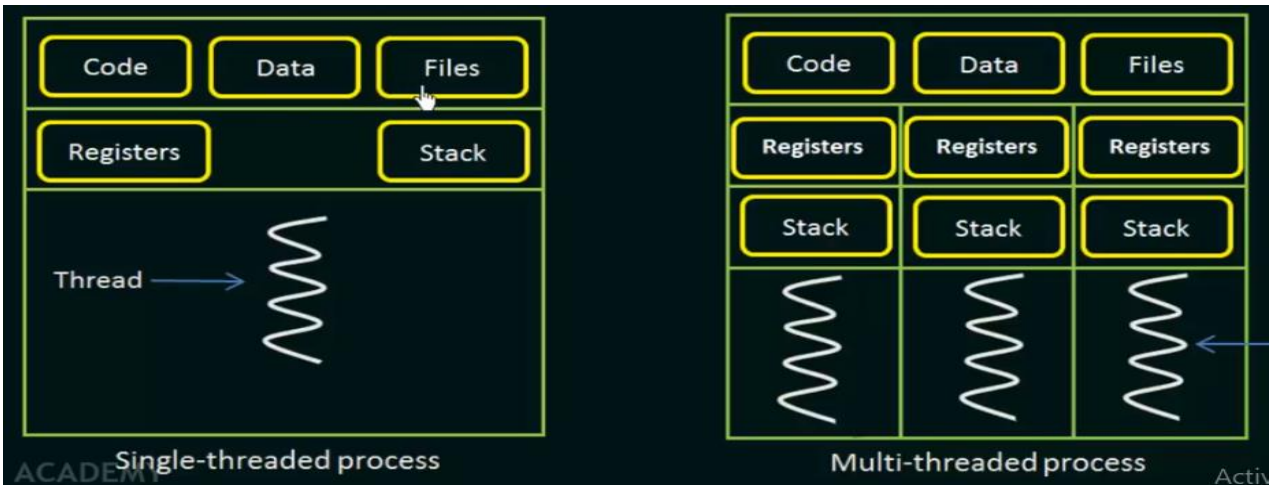
Def 2: A thread of execution is often regarded as the smallest unit of processing that a scheduler works on.



i. **A program counter-** is a register that manages the memory address of the instruction to be executed next…
ii. **A register set -** processor can be partitioned into different register classes.
iii. **A stack –** a mount of memory allocated to the stack in each thread.

It shares with other threads belonging to the same process its code section, data section, and other operating- system resources, such as open files and signals.

A traditional / heavyweight process has a single thread of control. If a process has multiple threads of control, it can perform more than one task at a time.



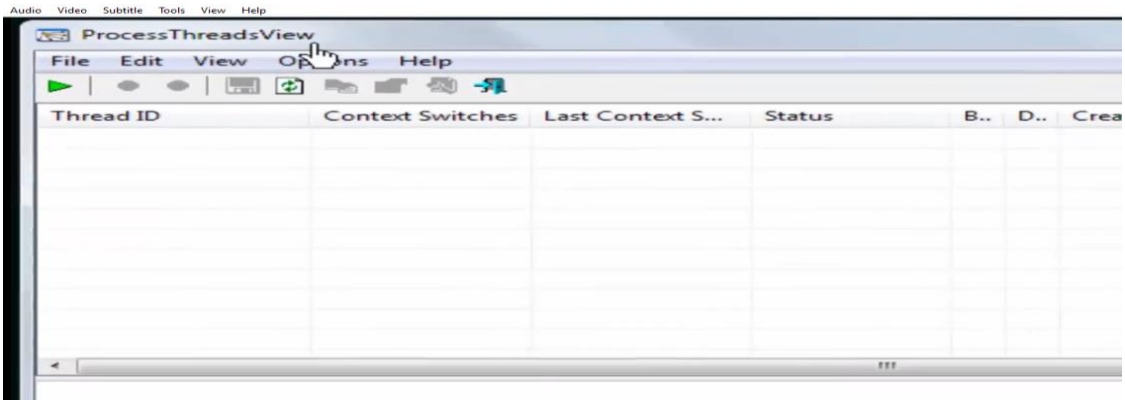


Fig 1 : process threads view

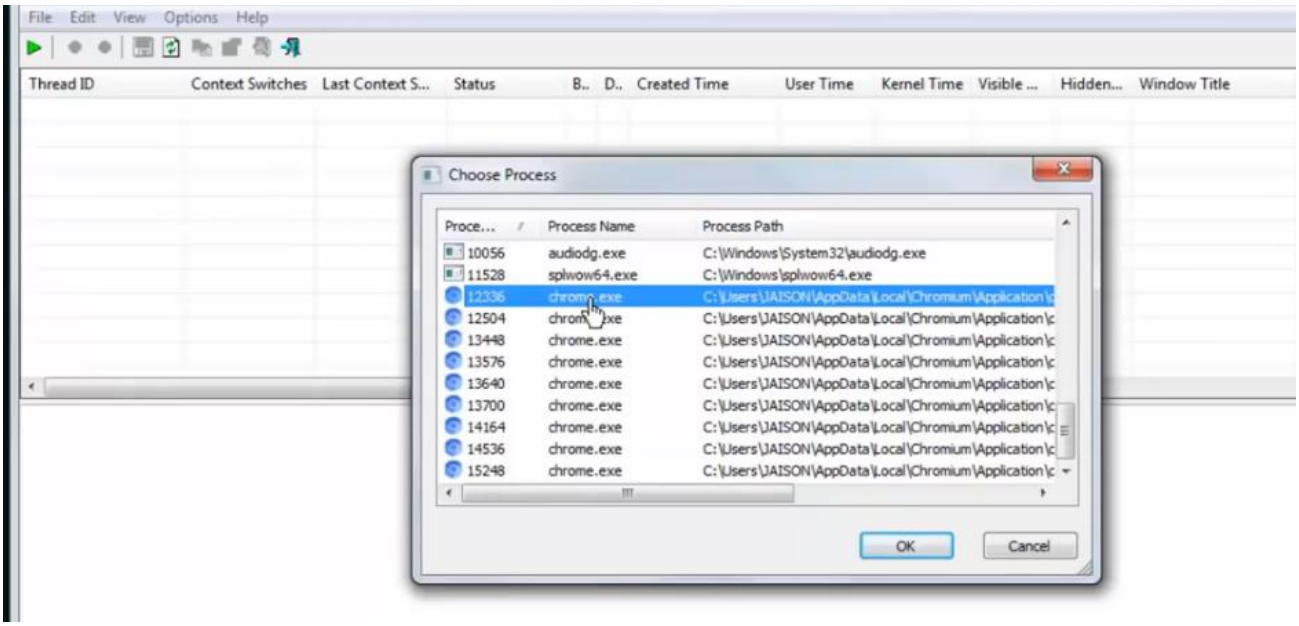


Fig 2: showing the different processes

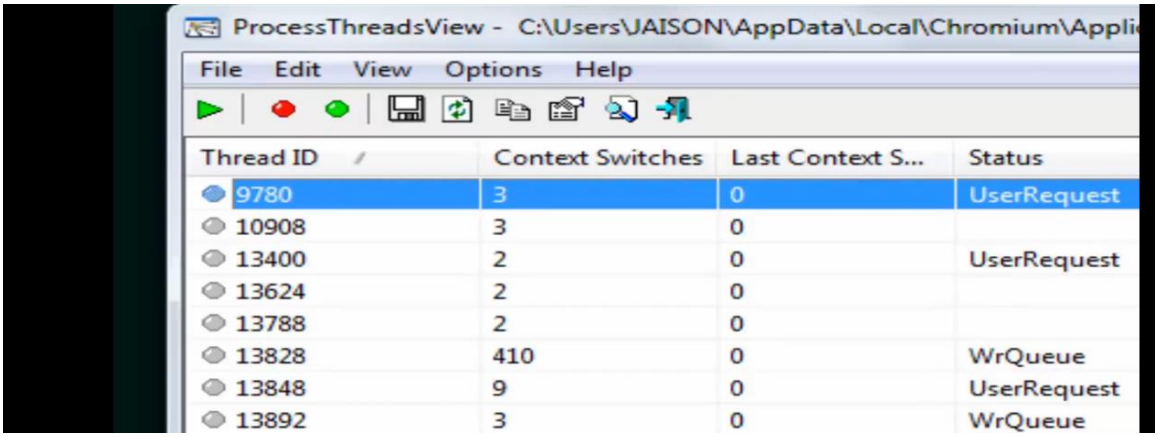
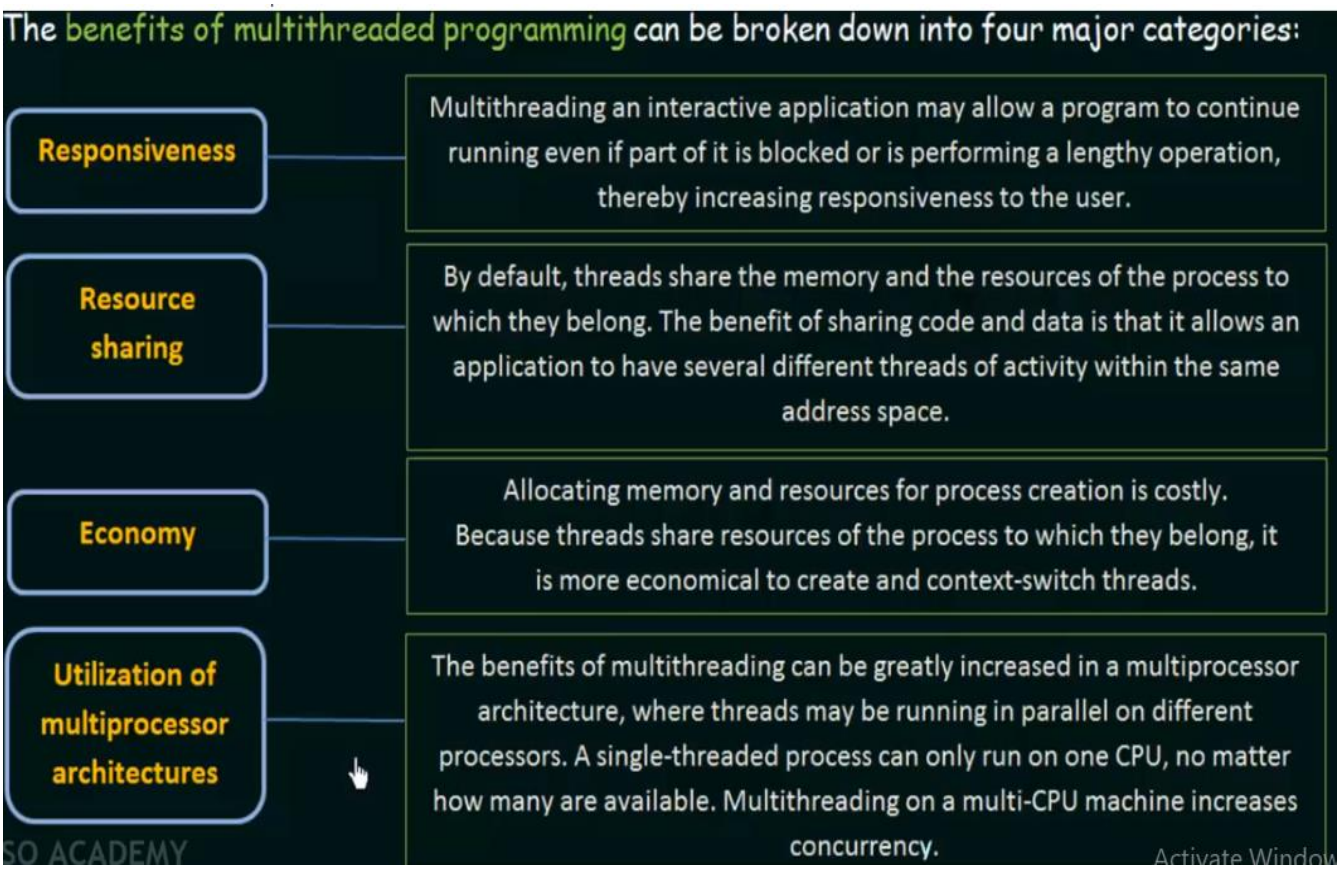


Fig 3: details concerning the threads from processor illustrated in Fig 2.



Threads are useful – Example applications

Example 1: A web serer

- It needs to handle several download requests over a short period
- Hence more efficient to create (and destroy) a single thread for each request
- Multiple threads can possibly be executing simultaneously on different processors

Example 2: A responsive user interface

- One area in which threads can be very helpful is in user-interface programs. These programs are usually centered around a loop of reading user input, processing it, and showing the results of the processing. The processing part may sometimes take a while to complete, and the user is made to wait during this operation. By placing such long operations in a separate thread to read, while having another thread to read user input, the program can be more responsive. It may allow the user to cancel the operation in the middle.

Thread Identification

Just as a process is identified through a **process ID**, a thread is identified by a **thread ID**. But interestingly, the similarity between the two ends here.

A **process ID** is unique across the system where as a thread ID is unique only in context of a single process.

A **process ID is an** integer value but the thread ID is not necessarily an integer value. It could well be a structure

A **process ID can be printed very easily** while a thread ID is not easy to print.

The above points give an idea about the difference between a process ID and thread ID.

Thread Creation

Normally when a program starts up and becomes a process, it starts with a default thread. So we can say that every process has at least one thread of control. **Thread ID** is represented by the type 'pthread_t'.

What are pthreads?

Historically, hardware vendors have implemented their own proprietary versions of threads. These implementations differed substantially from each other, making it difficult for programmers to develop portable threaded applications.

In order to take full advantage of the capabilities provided by threads, a standardized programming interface was required. for UNIX systems, **this interface has been specified by the IEEE POSIX 1003.1c standard (1995).** implementations which adhere to this standard are referred to as POSIX threads or Pthreads. Most hardware vendors now offer Pthreads in additional to their proprietary threads.

How to Create Threads ID in Linux (With a C Example Program)

We must include the pthread.h header file at the beginning of the script to use all the functions of the pthreads library.

How do I find thread ID?

The pthread_self () function is used to get the ID of the current thread. But if there are multiple threads, and one thread is completed, then that ID can be reused. Because it remains unchanged during its lifetime.

```
#include<stdio.h>
#include<stlib.h>
#include<pthread.h>
void* func(void*p)
{
printf("from the function,the thread id = %d\n"",pthread_self()); // get current thread id
pthread_exit(NULL);
return Null;
}
main() {
pthread_t thread; // declare thread
pthread_create (&thread,NULL,func,NULL);
print("from the main function,the thread id=%d\n", thread");
pthread_join(thread,NULL);// join with the main thread
}
```

Output

In function

Thread id =1

In main

Thread id =1

Void* func (void*p) – pointer which is not associated with any data types. It's a general purpose pointer. means function does not pass any argument.

Pthread_create (&thread, NULL, func, NULL) = creates a new thread in the calling process, if attr is NULL the default attributes shall be used.

Pthread_join – this is useful when you want to wait for thread/s to terminate before further processing in main thread.

Mutex and semaphore in systems administration

- **Read on mutex vs semaphore**

Javatpoint search mutex and semaphore

www.Guru99.com

- **when should we use semaphores and when should we use mutex?**
- **Read semaphore and mutex implementation using c programming**

Systems programming

COM 312: BSc Computer science

Lesson 6: updated notes.

Topics area coverage.

UNIX Thread management and synchronization

- Posix standards and its sub group
- What is thread? Importance of threads.
- Threading Concept in systems programming
- Thread ID calls
- Mutex and semaphore in systems administration

Posix standards and its sub group

Posix standards and its sub group

- **What is posix standards? "**Portability operating system interface**"** is the name of a family related standards specified by IEEE to define the application programming interface (API), along with shell utilities interface for the software compatible with variants of the UNIX operating system.

- Because many versions of UNIX exist today and each of them provides its own set of API functions, it is difficult for system developers to create applications that can be easily ported to different versions of UNIX.

- Some of **the subgroups** of POSIX are POSIX.1, POSIX.1b & POSIX.1c are concerned with the development of set of standards for system developers.

1. **POSIX. 1**
   - This committee proposes a standard for a base operating system API; this standard specifies APIs for the manipulating of files and processes.
   - It is formally known as IEEE standard 1003.1- 1990 and it was also adopted by the ISO as the international standard ISO/IEC 9945: 1: 1990.

2. POSIX.1b

- This committee proposes a set of standard APIs for a real time OS interface; these include IPC (interposes communication)
- This standard is formally known as IEEE standard 10034-1993.

3. Posix.1c

- This standard specifies multi-threaded programming interface. This is the newest POSIX standard.
- These standards are proposed for a generic OS that is not necessary be UNIX system.
- E.g VMS from Digital Equipment corporation, OS/2 from IBM, & Windows NT from Microsoft corporation are POSIX-compliant, yet they are not UNIX systems.
- To ensure a user program conforms to POSIX.1 standard the user should either define the manifested constant_ POSIX_SOURCE at the beginning of each source module of the program (before inclusion of any header) as; # define_POSIX_SOURCE OR specify the –D_ POSIX_SOURCE option to a C++ compiler(CC) in a compilation; % CC –D_ POSIX_SOURCE *.C
- POSIX.1b defines different manifested constant to check conformance of user program to that standard. The new macro is POSIX_C_SOURCE and its value indicates POSIX version to which a user program conforms. Its value can be:

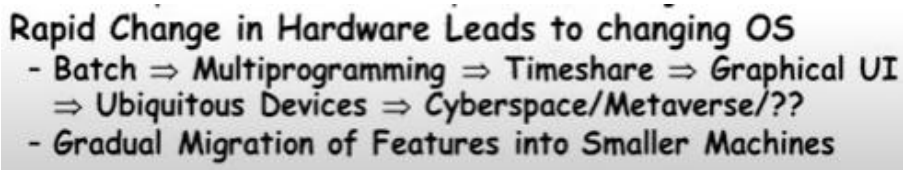
Threading Concept in programming.

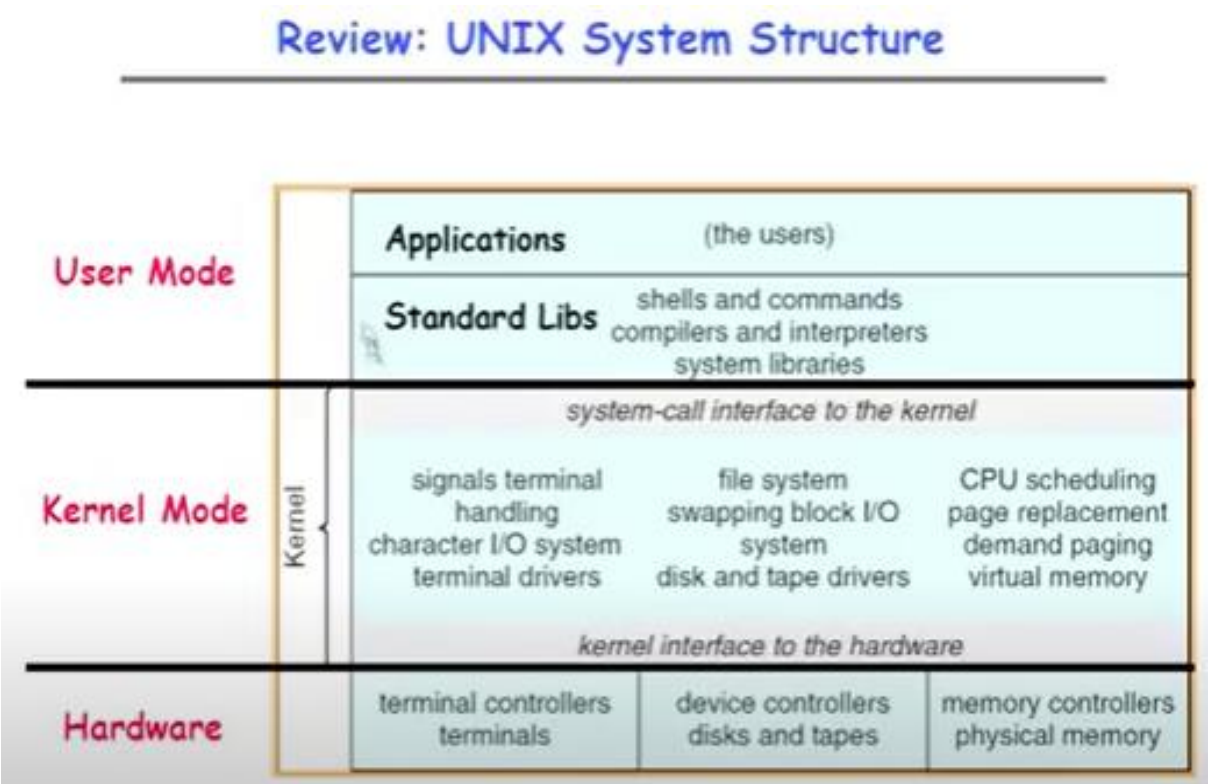
- **Threads** represent a software approach to improving performance of operating system through parallel computing by reducing the overhead thread is equivalent to a classical process.

- in modern programming **whenever a process has multiple tasks to perform independently of the others**.

What is a Thread?

- A thread is a path of execution within a process. A process can contain multiple threads.
  [Why study Threads in Computer science and Programming]





Why Multithreading?

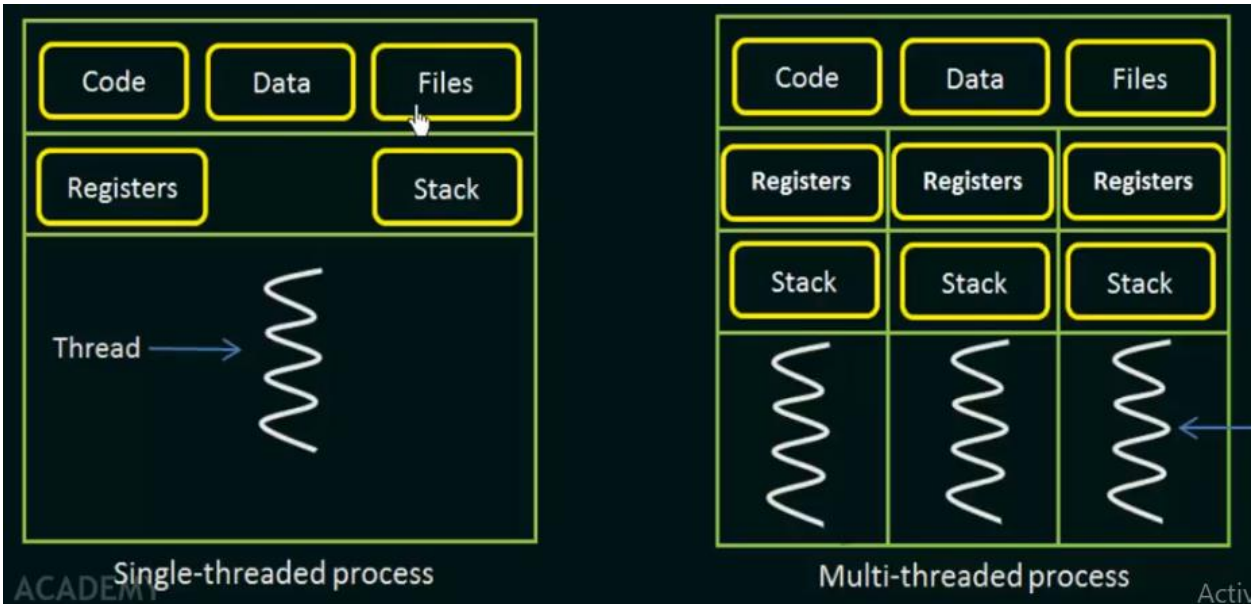
- A thread is also known as lightweight process.
- The idea is to achieve parallelism by dividing a process into multiple threads. **For example**, in a browser, multiple tabs can be different threads. MS Word uses multiple threads: *one thread to format the text*, another thread *to process inputs*, etc. More advantages of multithreading are discussed below

Process vs Thread?

- The primary difference is that

- **threads** within the same process *run in a shared memory space,*
- while **processes** run *in separate memory spaces.*

- Threads are not independent of one another like processes are, and as a result threads share with other threads their code section, data section, and OS resources (like open files and signals).
- **But, like process,** a thread has its own program counter (PC), register set, and stack space.



This is the high-level definition of threads, let's take a slightly deeper look into how threads are stored in memory.

Thread Control Block

Thread ID
Thread state
CPU information : Program counter Register contents
Thread priority
Pointer to process that created this thread
Pointer(s) to other thread(s) that were created by this thread

Thread ID – A unique identifier that gets generated by the operating system when a thread gets created.

Thread States – State of the thread that changes as the thread progresses throughout the system.

CPU information – This includes everything that the OS needs to know about the thread such as Program Count and Register Contents.

Thread Priority – It indicates the weight (or priority) of the thread over other threads which helps the thread scheduler to determine which thread should be selected next from the READY queue.

Process Pointer – A pointer to the process that started this thread.

Thread Pointer(s) – A pointer or list of pointers to other threads that were created by the current thread. Now that we know

How do multiple threads speed up my code?

Let's take a web server as an example.

- If the server created a separate process for every HTTP request, we would wait forever for our page to load. The reason is that creating a new process is an expensive procedure for the operating system.

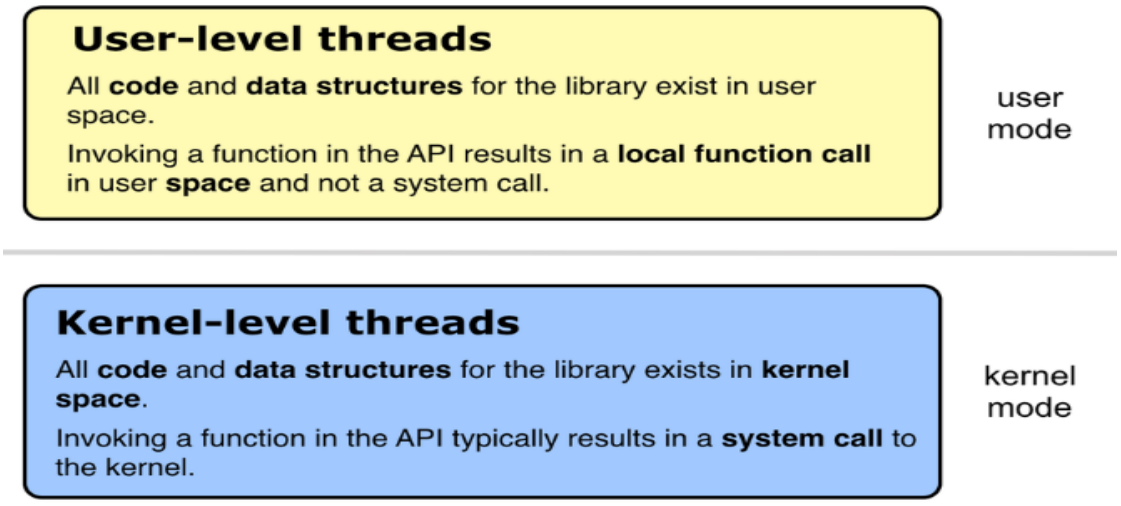
- Instead of creating a new process, it would be more efficient to create a new thread.

- But sometimes this wouldn't speed up your application. If your lacking program uses a lot of computing power, then utilizing multiple processes would be more beneficial.

- On the other hand, if you're facing an Input-Ouput (IO) problem, then multithreading will help you.

Next, let's look at different types of threads and their relationships to each other.

Thread Types



What are Pthreads/posix threads?

- Pthreads is a C/C++ library used to manage threads that are based on the POSIX standard.

The methods in the Pthreads library can be categorized into four groups:

1. **Thread Management –** Routines that work directly on threads - creating, detaching, joining, etc. They also include functions to set/query thread attributes (joinable, scheduling etc.)
2. **Mutexes –** Routines that deal with synchronization, called a "mutex", which is an abbreviation for "mutual exclusion". Mutex functions provide for creating, destroying, locking and unlocking mutexes. These are supplemented by mutex attribute functions that set or modify attributes associated with mutexes.
3. **Condition variables –** Routines that address communications between threads that share a mutex. Based upon programmer specified conditions. This group includes functions to create, destroy, wait and signal based

upon specified variable values. Functions to set/query condition variable attributes are also included.

4. **Synchronization –** Routines that manage read/write locks and barriers.

Identifying a thread

Each thread identified by an ID, which is known as Thread ID. Thread ID is quite different from Process ID. *A Thread ID is unique in the current process*, while *a Process ID is unique across the system.*

Thread ID is represented by type **pthread_t**

Header file(s)

The header file which needs to be included to access thread functions

```
#include<pthread.h>
```

Creating a Thread (pthread_create)

pthread_create is the function of pthread.h header file, which is used to create a thread. The syntax and parameters details are given as follows:

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
   void *(*start_routine) (void *), void *arg);
```

pthread_t *thread

It is the pointer to a pthread_t variable which is used to store thread id of new created thread.

const pthread_attr_t *attr

It is the pointer to a thread attribute object which is used to set thread attributes, NULL can be used to create a thread with default arguments.

void *(*start_routine) (void *)

It is the pointer to a thread function; this function contains the code segment which is executed by the thread.

void *arg

It is the thread functions argument of the type void*, you can pass what is necessary for the function using this parameter.

int (return type)

If thread created successfully, return value will be 0 (Zero) otherwise pthread_create will return an error number of type integer.

Consider the example using thread

```
#include <stdio.h>
#include <unistd.h>  //Header file for sleep(). man 3 sleep for details.
#include <pthread.h>

void *myThreadFun()
{

    sleep(5);
    printf("Printing Thread \n");
    return NULL;
}

int main()
{
    pthread_t thread_id;//thread declaration of type 'pthread_t'
    printf("Before Thread\n");
    pthread_create(&thread_id, NULL, myThreadFun, NULL); //thread is created
    pthread_join(thread_id, NULL); //process waits for thread to finish
    printf("After Thread-common message\n");

}
```

Output of P Thread

```
File  Edit  View  Search  Terminal  Help
[centos@localhost osprogs]$ gedit thread0.c
[centos@localhost osprogs]$ gedit thread0.c
[centos@localhost osprogs]$ gcc thread0.c -lpthread
[centos@localhost osprogs]$ ./a.out
Before Thread
Printing Thread
After Thread-common message
[centos@localhost osprogs]$
```

What is process synchronization in operating system?

- **Synchronization** - *is the way by which processes that share the same memory space are managed in an operating system.*
- When more than two processes have access to the same data or resources at the same time it can cause data inconsistency so to remove this data inconsistency processes should be synchronized with each other.

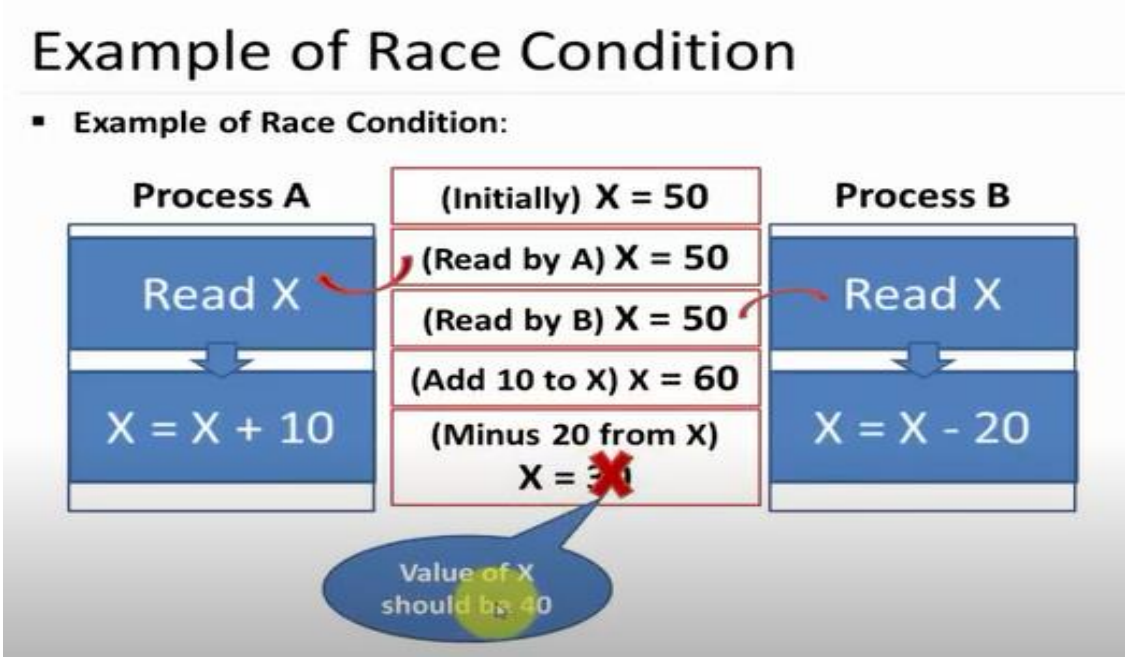


In the above picture, we take an example of a bank account that has a current balance of 500 and there are two users which have access to that account**. User 1 and User 2 both are trying to access the balance.** If process 1 is for **withdrawal** and process 2 is for **checking the balance** both will occur at the same time, then user 1 might get the wrong current balance*. To avoid this kind of data inconsistency process synchronization in os is very helpful.*

Two important concepts that are related to process synchronization i.e. Race Condition and Critical Section

Race condition

- A **race condition** is a condition when there are many processes and every process shares the data with each other and accessing the data concurrently, and the output of execution depends on a particular sequence in which they share the data and access.



Critical Section

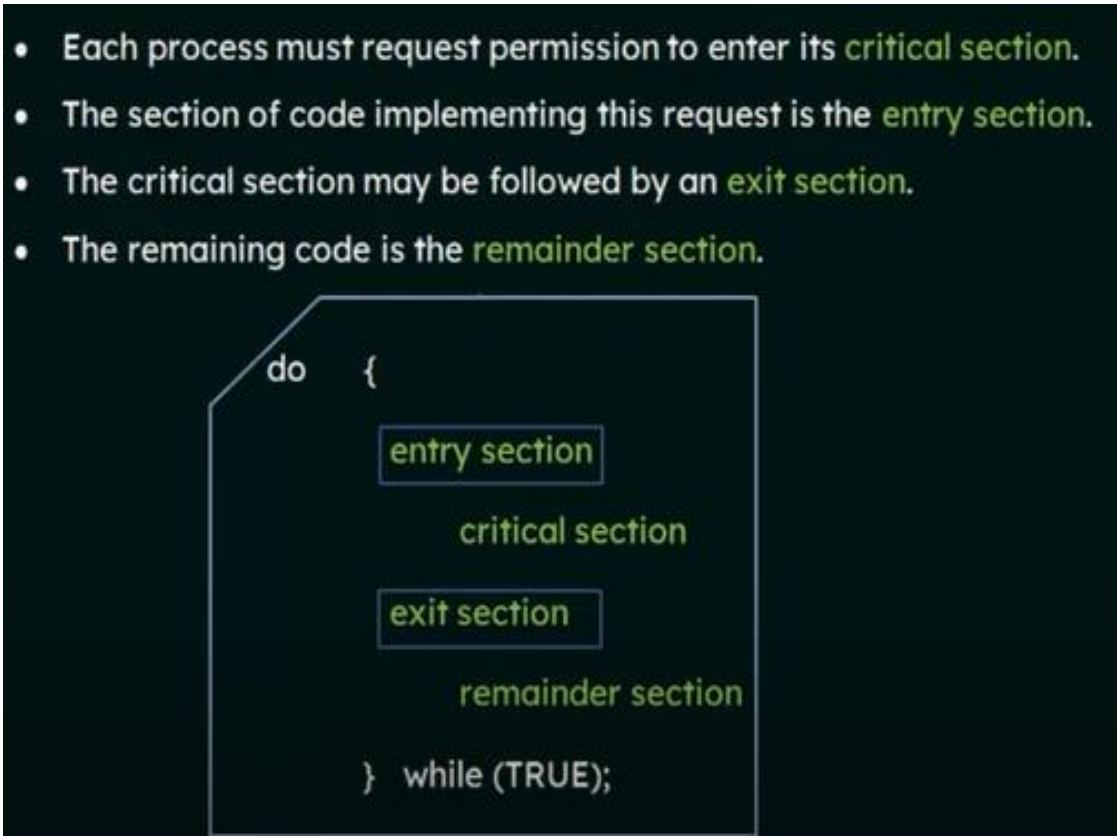
- The term "**critical section**" refers to a code segment that is accessed by several programs. The critical section includes shared variables or resources that must be synchronized in order to ensure data consistency.

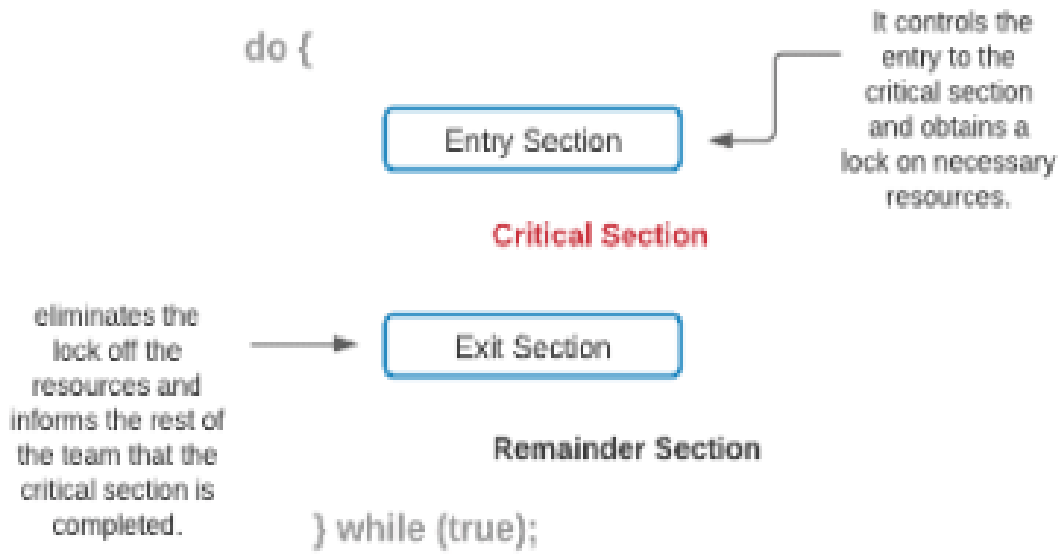
- **Critical Section** *is the part of a program which tries to access shared resources.* That resource may be any resource in a computer like a memory location, Data structure, CPU or any IO device.

- The critical section cannot be executed by more than one process at the same time; operating system faces the difficulties in allowing and disallowing the processes from entering the critical section.

- The **critical section problem** is used to design a set of protocols which can ensure that the Race condition among the processes will never arise.

General structure of a typical process





Solutions to the critical section problem

Any solution to the critical section problem must satisfy the following requirements:

1. **Mutual exclusion:** When one process is executing in its critical section, no other process is allowed to execute in its critical section.
2. **Progress**: When no process is executing in its critical section, and there exists a process that wishes to enter its critical section, it should not have to wait indefinitely to enter it.
3. **Bounded waiting:** There must be a bound on the number of times a process is allowed to execute in its critical section, after another process has requested to enter its critical section and before that request is accepted.

Most solutions to the critical section problem utilize locks implemented on software. The solutions include:

1. **test_and_set:** Uses a shared boolean variable lock and the test_and_set instruction that executes atomically and sets the passed parameter value to true.

2. compare_and_swap: Same as test_and_set, except that the passed parameter value is set to true if it is equal to expected in the parameter of the compare_and_swap instruction.

3. **Mutex locks**: Software method that provides the acquire () and release () functions that execute atomically.

4. **Semaphores:** More sophisticated methods that utilize the wait () and signal () operations that execute atomically on Semaphore S, which is an integer variable.

5. **Condition variables:** Utilizes a queue of processes that are waiting to enter the critical section.

Uses of Critical section

1. **Critical sections in data structures**

The read-write conflicting variables are divided between threads, with a copy in each thread. Data structures such as linked lists, trees, and hash tables have connected data variables that cannot be divided across threads, making parallelism difficult to implement.

To avoid this, one technique is to keep the complete data structure in a critical section and handle just one action at a time. Another option is to lock the critical section node in use so that other actions do not utilize the same node. As a result, using the critical section assures that the code produces the intended results.

2. **Critical sections in computer networking**

In computer networking, critical sections are also required. Data may not arrive in an ordered fashion when it reaches network sockets. Let's pretend that program X on the computer needs to take data from the socket, reorganize it, and check for any errors. No other application should access the same socket for that data while this program is working on it. As a result, the socket's data is secured by a critical section, allowing program X to use it exclusively.

3. **Kernel-level critical sections**

Critical section often blocks thread and process migration across processors, as well as interruptions and other processes and threads from preempting processes and threads. Nesting is common in critical parts. Multiple essential parts can be accessed and exited at little cost because of nesting.

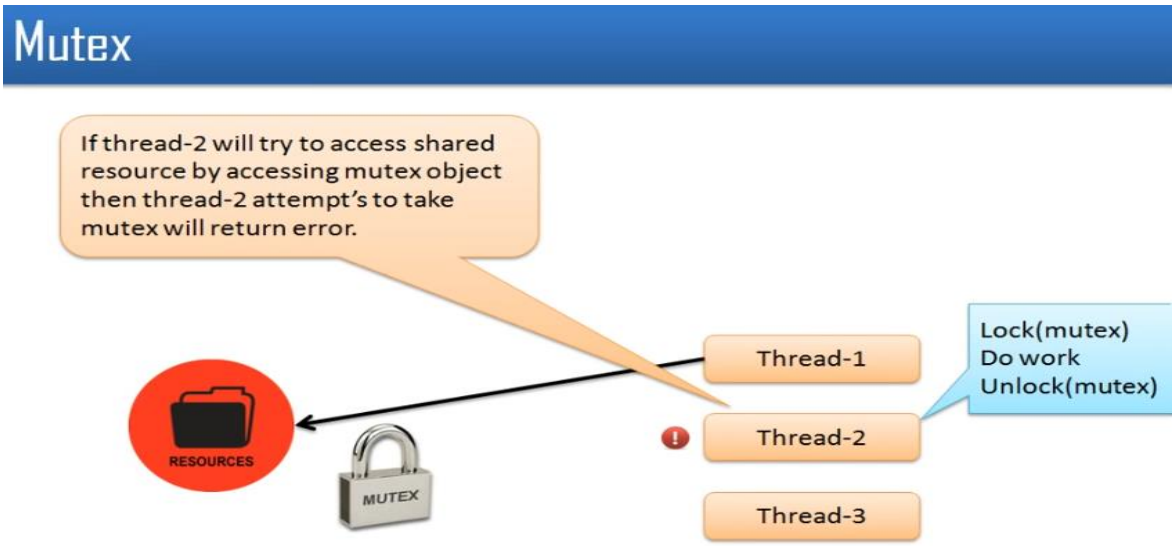
Because the critical sections may only operate on the processor into which they are inserted, synchronization is only needed inside the executing processor. This enables important portions to be entered and exited at virtually no cost. There is no need for inter-processor synchronization. All that is required is instruction stream synchronization. Most CPUs offer the necessary synchronization simply by stopping the current execution state.

4. The usage of critical sections as a long-term locking primitive is not recommended. The software lockout problem is based on kernel-level critical sections.

Mutex and Semaphore

Mutex

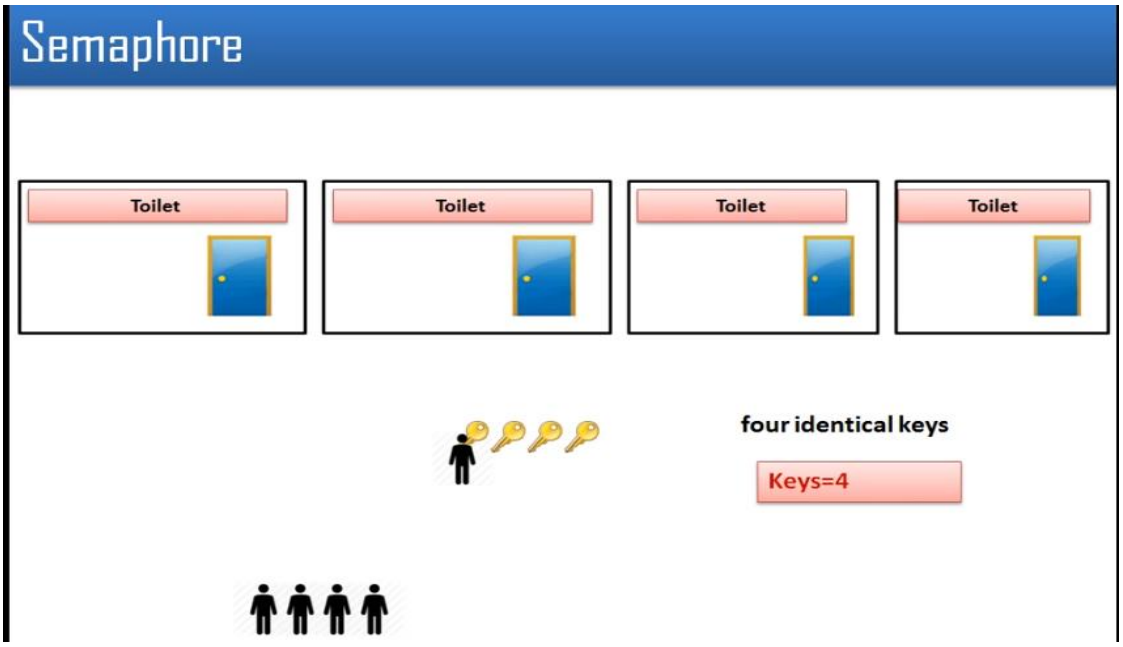


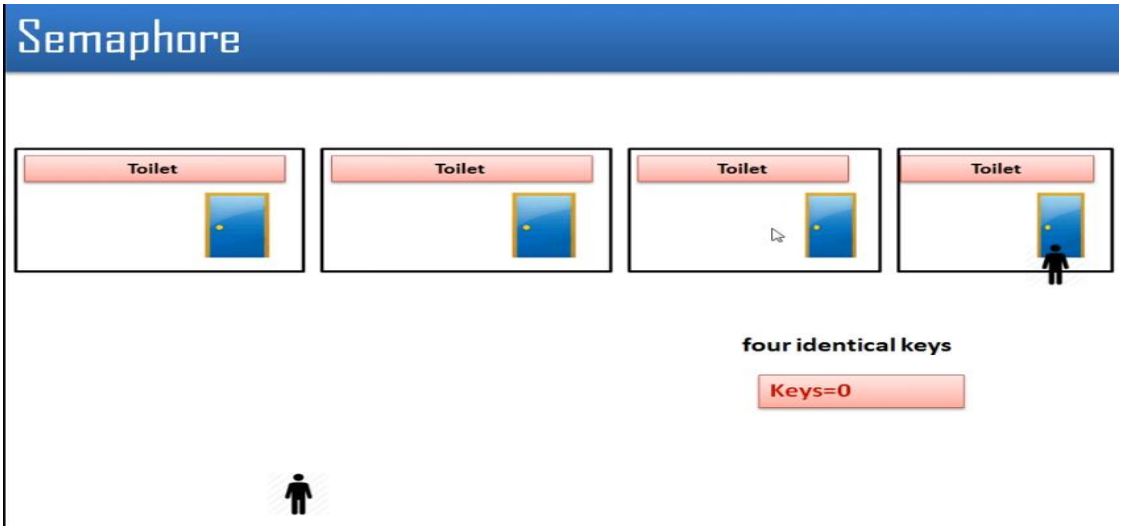


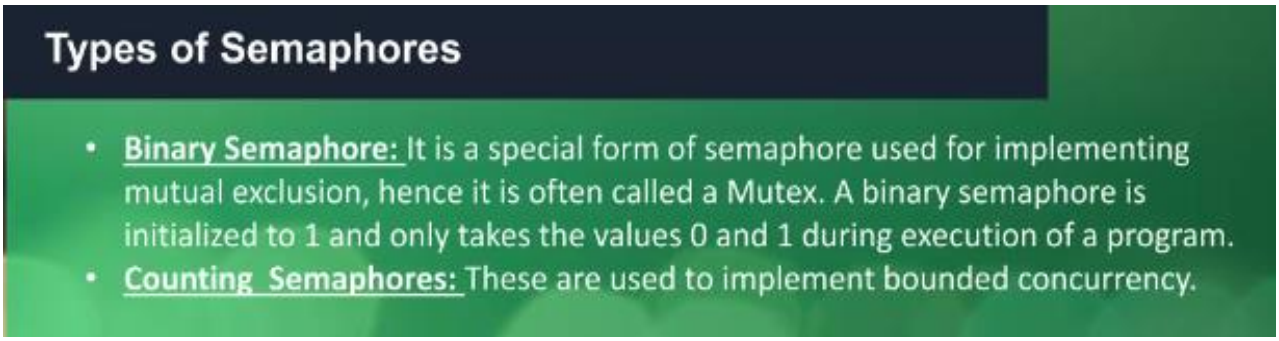
Semaphore

Semaphore is a non-negative variable that is shared across threads. It's a different algorithm or solution to the problem of the critical section. It's a thread that's waiting for a semaphore, which can be signaled by another thread.

For process synchronization, it employs two atomic operations: 1) wait() and 2) signal().







PRACTICE PROBLEMS BASED ON BINARY SEMAPHORES IN OS-

Problem-01:

Each process Pi, i = 1, 2, …, 9 is coded as follows-

repeat

P(mutex)

{ Critical Section }

V(mutex)

forever

QUIZ

The code for P10 is identical except that it uses V(mutex) in place of P(mutex). What is the largest number of processes that can be inside the critical section at any moment?

1

2

3

None of these

Solution-

Code for Processes P1, P2, … , P9

repeat

P(mutex)

{ Critical Section }

V(mutex)

forever

Code for Process P10

repeat

V(mutex)

{ Critical Section }

V(mutex)

forever

Consider mutex is initialized with 1.

Then-

All the 10 processes may be present inside the critical section at the same time.

If there would have been more number of processes, then they could also be present inside the critical section at the same time.

The explanation is as follows-

Explanation-

Scene-01:

Initially, process P1 arrives.

It performs the wait operation on mutex and sets its value to 0.

It then enters the critical section.

Scene-02:

Consider the processes P2, P3, … , P9 arrives when process P1 is executing the critical section.

All the other processes get blocked and are put to sleep in the waiting list.

Scene-03:

Process P10 arrives.

It performs the signal operation on mutex.

It selects one process from the waiting list say process P2 and wakes it up.

Process P2 can now enter the critical section (P1 is already present).

Now, process P10 enters the critical section (P1 and P2 are already present).

After executing critical section, during exit, it again performs the signal operation on mutex.

It selects one process from the waiting list say process P3 and wakes it up.

Process P3 can now enter the critical section (P1 and P2 are already present).

Process P10 may keep on executing repeatedly.

It wakes up 2 processes each time during its course of execution.

In this manner, all the processes blocked in the waiting list may get entry inside the critical section.

Thus, Option (D) is correct.

Systems programming lesson 7

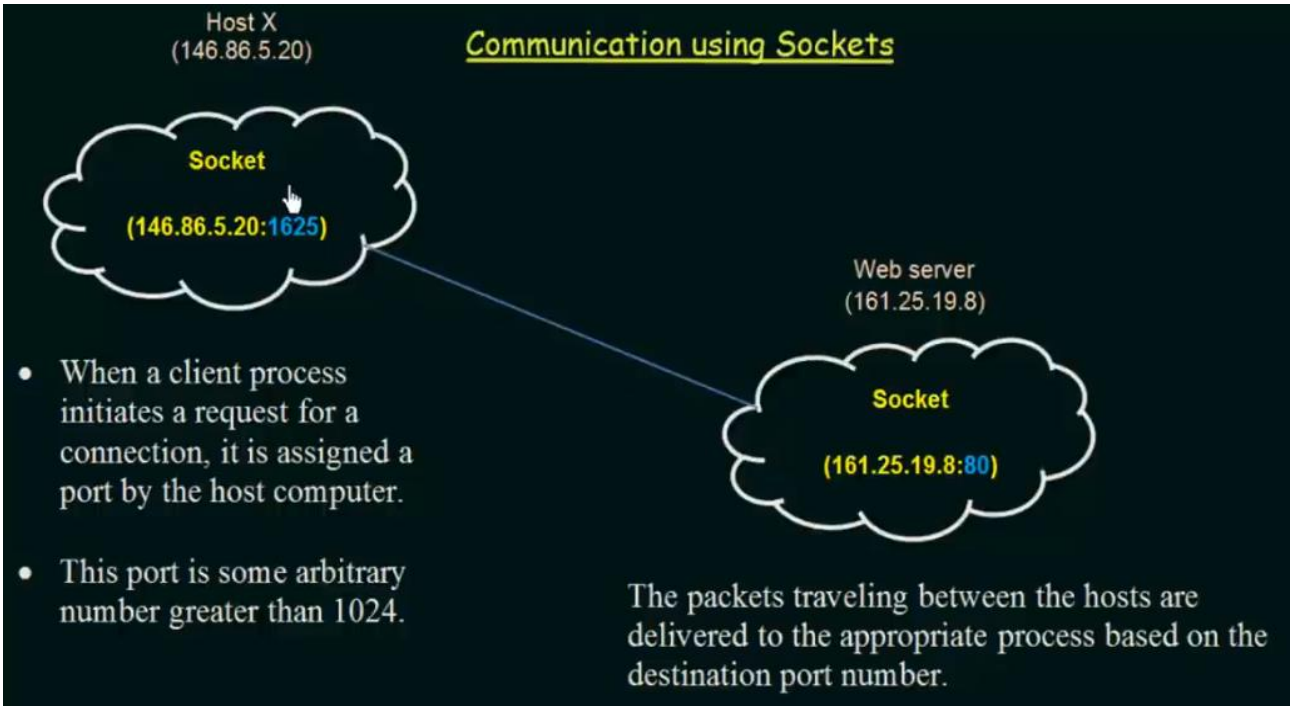
COM 312: BSc Computer science

Topics area coverage.

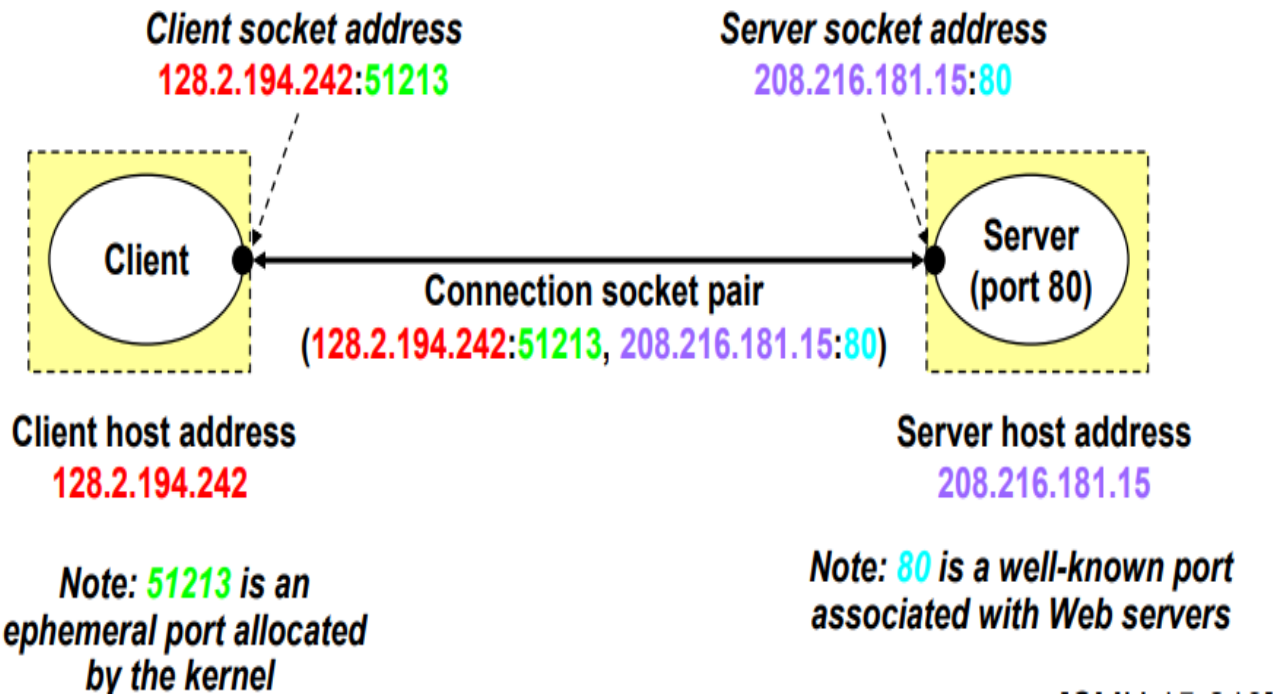
- **Socket programming in unix**
- **Socket system call APIs**

Socket programming

- **Socket is a virtual endpoint of any kind of network communication done between two hosts over in a network.** Sockets can be used in many languages like Java, C++, C etc.
- The socket API on Linux is similar to BSD/UNIX sockets from which it has evolved. Although over time the API has become slightly different at few places and now the newer official standard is POSIX sockets API which is same as BSD sockets.
    - A socket is identified by an IP address concatenated with a port number.
    - The server waits for incoming client requests by listening to a specified port. Once a request is received, the server accepts a connection from the client socket to complete the connection.
    - Servers implementing specific services (such as telnet, ftp, and http) listens to well-known ports (a telnet server listens to port 23, an ftp server listens port 21 and a web or http, server listens to port 80).
    - All ports below 1024 are considered well known; we can use them to implement standard services


Host X
(146.86.5.20)
Communication using Sockets
Socket
(146.86.5.20:1625)
Web server
(161.25.19.8)
Socket
(161.25.19.8:80)
When a client process initiates a request for a connection, it is assigned a port by the host computer.
This port is some arbitrary number greater than 1024.
The packets traveling between the hosts are delivered to the appropriate process based on the destination port number.

Example 2


Client socket address
128.2.194.242:51213
Server socket address
208.216.181.15:80
Client
Server
(port 80)
Connection socket pair
(128.2.194.242:51213, 208.216.181.15:80)
Client host address
128.2.194.242
Server host address
208.216.181.15
Note: 51213 is an ephemeral port allocated by the kernel
Note: 80 is a well-known port associated with Web servers

How to create a socket?

To create a socket, we can use the Socket function. Here is the sample code.

socket_tutorial.c

```c
#include<stdio.h>
#include<sys/socket.h>

int main(int argc , char *argv[])
{
    int socket_desc;
    socket_desc = socket(AF_INET , SOCK_STREAM , 0);

    if (socket_desc == -1)
    {
        printf("Could not create socket");
    }

    return 0;
}
```

Code explanation

socket () creates a socket and returns a descriptor which can be used in other functions.

```c
int main(int argc , char *argv[])
```

- **argc** (ARGument Count) is int and stores number of command-line arguments passed by the user including the name of the program.
- The value of argc should be non-negative.
- **Arg v** (ARGument Vector) is array of character pointers listing all the arguments.

The above code will create a socket with following properties.

- **Address Family: AF_INET (**IP Version 4**)**
- **Type: SOCK_STREAM (**TCP Protocol**)**
- **Protocol: 0 (**Default IP Protocol**)**
- Instead of using SOCK_STREAM type of sockets there is another type called SOCK_DGRAM which indicates the UDP protocol.

Types of Sockets:

There are two types of Sockets: the datagram socket and the stream socket.

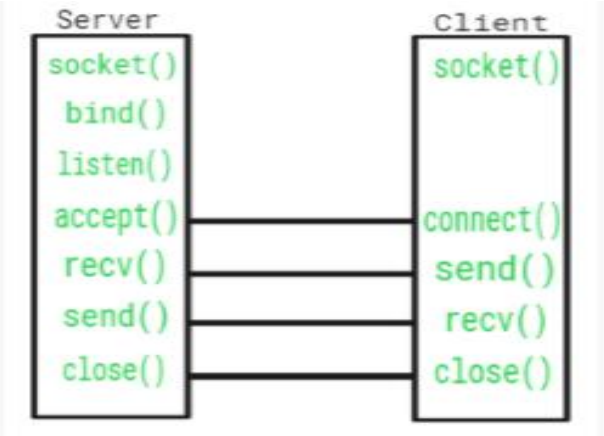
Datagram Socket:

This is a type of network which has connection less point for sending and receiving packets. It is similar to mailbox. The letters (data) posted into the box are collected and delivered (transmitted) to a letterbox (receiving socket).

Stream Socket

In Computer operating system, a stream socket is type of interposes communications socket or network socket which provides a connection-oriented, sequenced, and unique flow of data without record boundaries with well-defined mechanisms for creating and destroying connections and for detecting errors. It is similar to phone. A connection is established between the phones (two ends) and a conversation (transfer of data) takes place.

example 1 **how** network socket uses system calls API to communicate



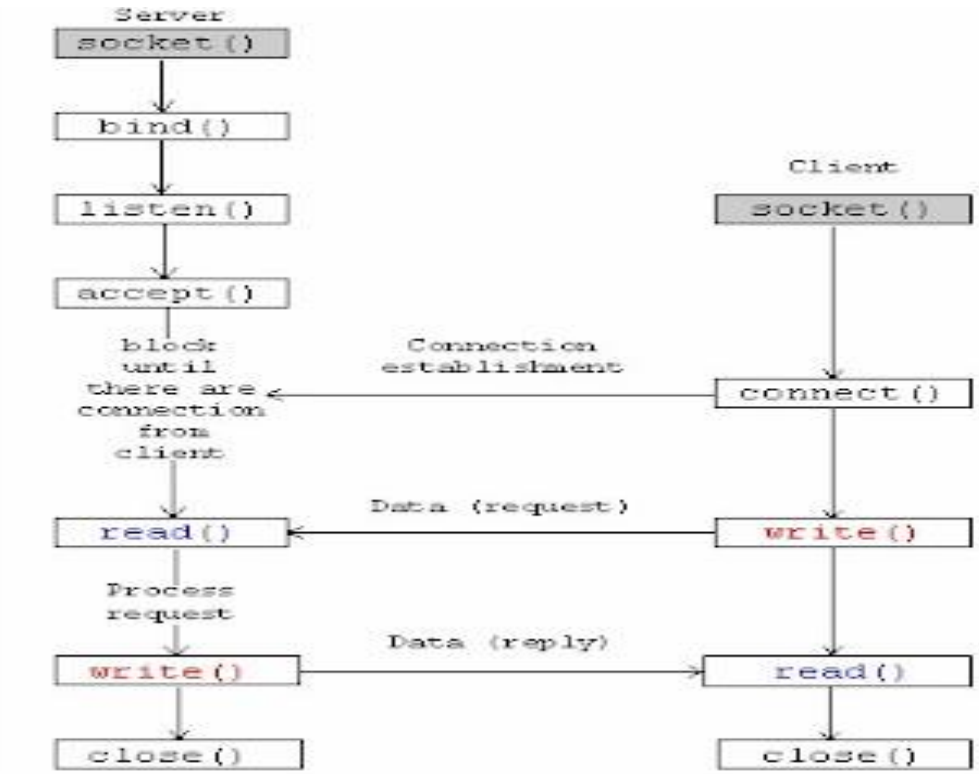
Function Call	Description
Create ()	To create a socket
Bind ()	It's a socket identification like a telephone number to contact
Listen ()	Ready to receive a connection
Connect ()	Ready to act as a sender

Accept ()	Confirmation, it is like accepting to receive a call from a sender
Write ()	To send data
Read ()	To receive data
Close ()	To close a connection

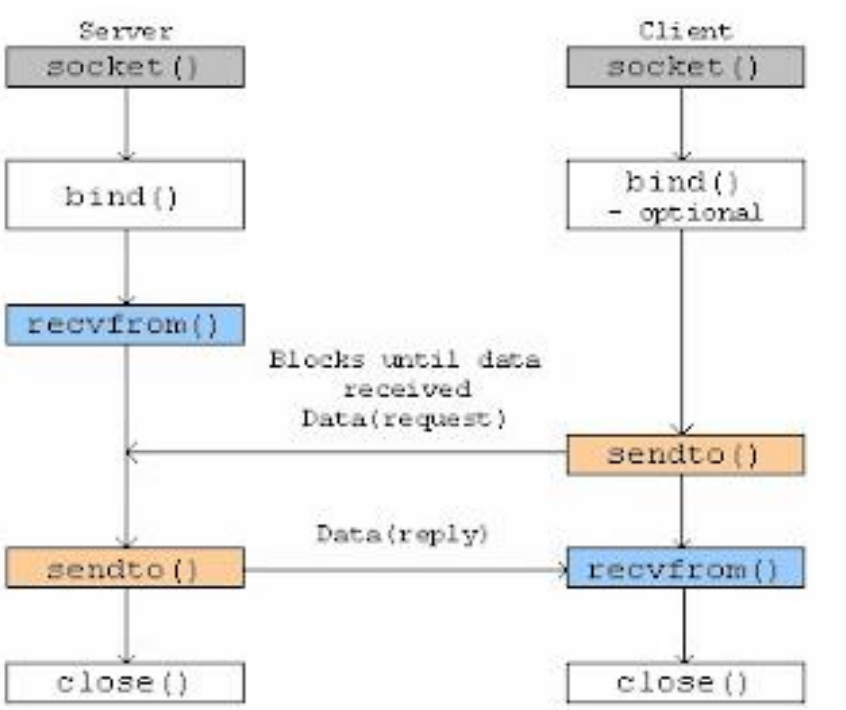
how client and server processes across a network communicate using **socket API**

1. **client/server relationship of the socket APIs for connection-oriented protocol (TCP)**
2. **client/server relationship of the socket APIs for a connectionless protocol (UDP)**

client/server relationship of the socket APIs for connection-oriented protocol (TCP)



client/server relationship of the socket APIs for a connectionless protocol (UDP)



Port vs. Socket

We now know the basics of port and socket. Let's see the differences between a port and a socket:

Port	Socket
Port specifies a number that is used by a program in a computer.	A socket is a combination of IP address and port number.
A program running on different computers can use the same port number. Hence port numbers can't be used to identify a computer uniquely.	It identifies a computer as well as a program within the computer uniquely.
Port number is used in the transport layer.	Sockets are involved in the application layer. A socket is an interface between the transport and application layer.
Port uses a socket to drop the data to a correct application.	A server and a client uses a socket to keep an eye on the data request and responses.

How Do Sockets Work?

Sockets simply provide the actual hardware for moving data around. TCP-based sockets are called stream sockets, where all data will arrive in order. UDP-based sockets are datagram sockets, where order (or even delivery) isn't guaranteed. There are also raw sockets, which don't have any restrictions, and are used for implementing different protocols and utilities that need to inspect low-level network traffic, like **Wireshark.**

Sockets usually still use TCP or UDP, **as they aren't anything special other than a fancy pipe within the kernel.** TCP and UDP are transport protocols that define how data gets from place to place but don't really care about what the data is. TCP and UDP provide the platform for most other protocols like FTP, SMTP, and RDP, which operate at higher levels.

It is possible for an application to use a slightly different implementation of TCP; stream sockets use the SOCK_STREAM protocol, which is what TCP also uses for transport almost all the time, and while they're basically interchangeable, they're technically slightly different. Though this is low-level stuff and isn't really something you'll have to worry about, just know that most traffic sent through UNIX domain sockets is TCP- or UDP-based, or at least quite similar to it, and TCP sent over UNIX domain sockets is faster than TCP over network interfaces like ports.

Socket Use in Practice

The biggest example of this is Redis, an extremely fast key-value store that operates entirely within memory. Redis is frequently used on the same server that's accessing it, so you'll usually be able to use sockets. At such low levels and with how fast Redis is, sockets provide a 25%-performance boost in some synthetic benchmarks.

If you're connecting to a MySQL database, you can also use a socket. Usually you'd connect to host: port from a remote system, but if you're connecting to a database on

the same server (for example, an REST API accessing a database), you can use sockets for a speedup. This won't affect normal use, but is very noticeable when under load, over 20% on a high end 24 core with 128 concurrent users and a million queries per second. Whether or not you'll see a benefit from sockets is a different story, but at that point you'll likely want to be looking into replication and load balancing anyway.